

Biomedical Computer Vision

Introduction to Deep Learning

March 31st, 2026

Virginia Tasso
virginia.tasso@polimi.it



Syllabus:

17.03

- Data dimensionality, image formats
- Image preprocessing

19.03

- Introduction to convolution
- Padding, stride, pooling

24.03

- Convolution-based operation: filtering (denoising, sharpening, edge detection)

27.03

- Introduction to segmentation: thresholding, masking, k-means...
- Post-processing: connected components, erosion, dilation...

31.03

- Introduction to Deep Learning
- Fully Connected Neural Networks, Convolutional Neural Networks

02.04

- NN training and regularization (dropout, weight decay...)

14.04

- Image segmentation with Deep Learning
- UNet

16.04

- CNN and others for other tasks (generation, denoising...)
- Review of the state of the art

Syllabus:

17.03

- Data dimensionality, image formats
- Image preprocessing

19.03

- Introduction to convolution
- Padding, stride, pooling

24.03

- Convolution-based operation: filtering (denoising, sharpening, edge detection)

27.03

- Introduction to segmentation: thresholding, masking, k-means...
- Post-processing: connected components, erosion, dilation...

31.03

- Introduction to Deep Learning
- Fully Connected Neural Networks, Convolutional Neural Networks

02.04

- NN training and regularization (dropout, weight decay...)

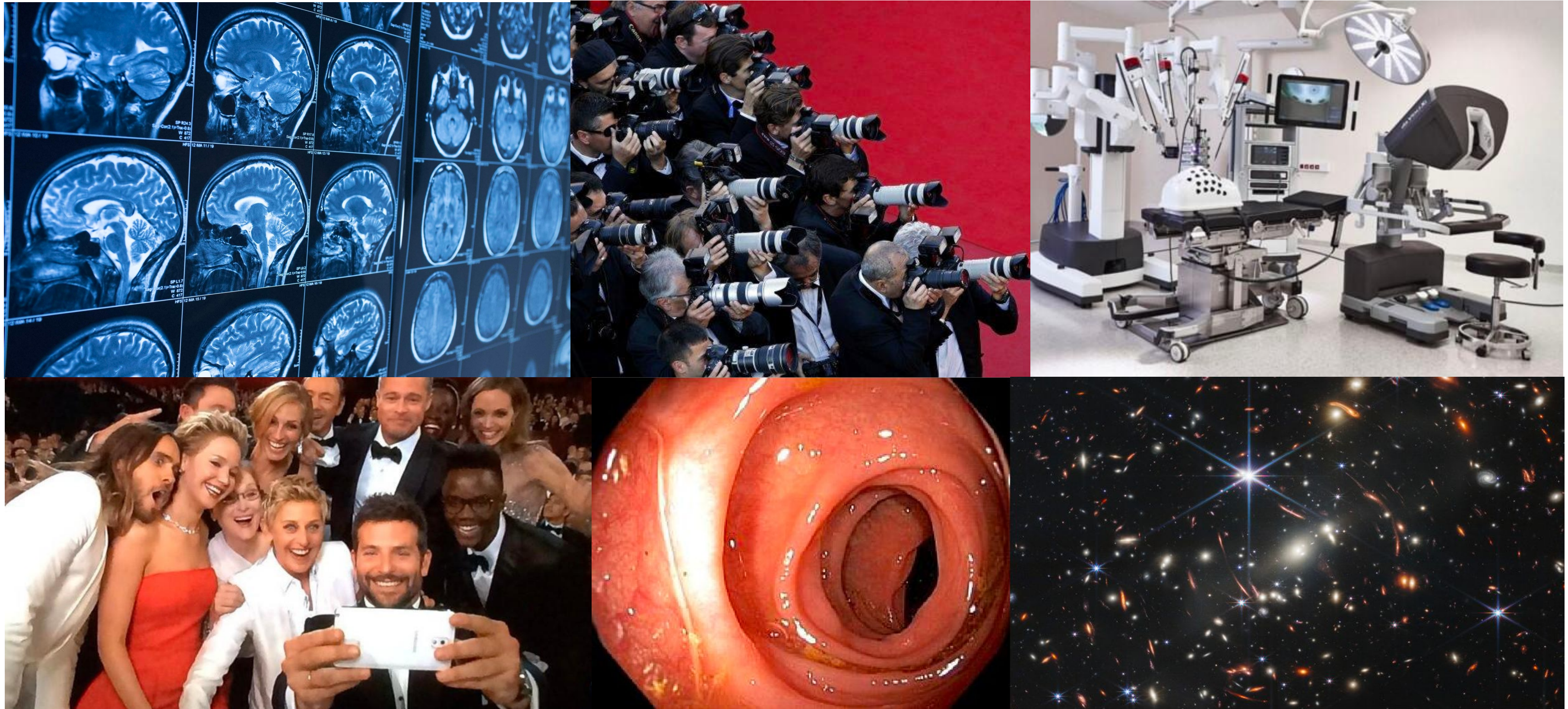
14.04

- Image segmentation with Deep Learning
- UNet

16.04

- CNN and others for other tasks (generation, denoising...)
- Review of the state of the art

Big Image Data



Manual analysis vs automatic analysis

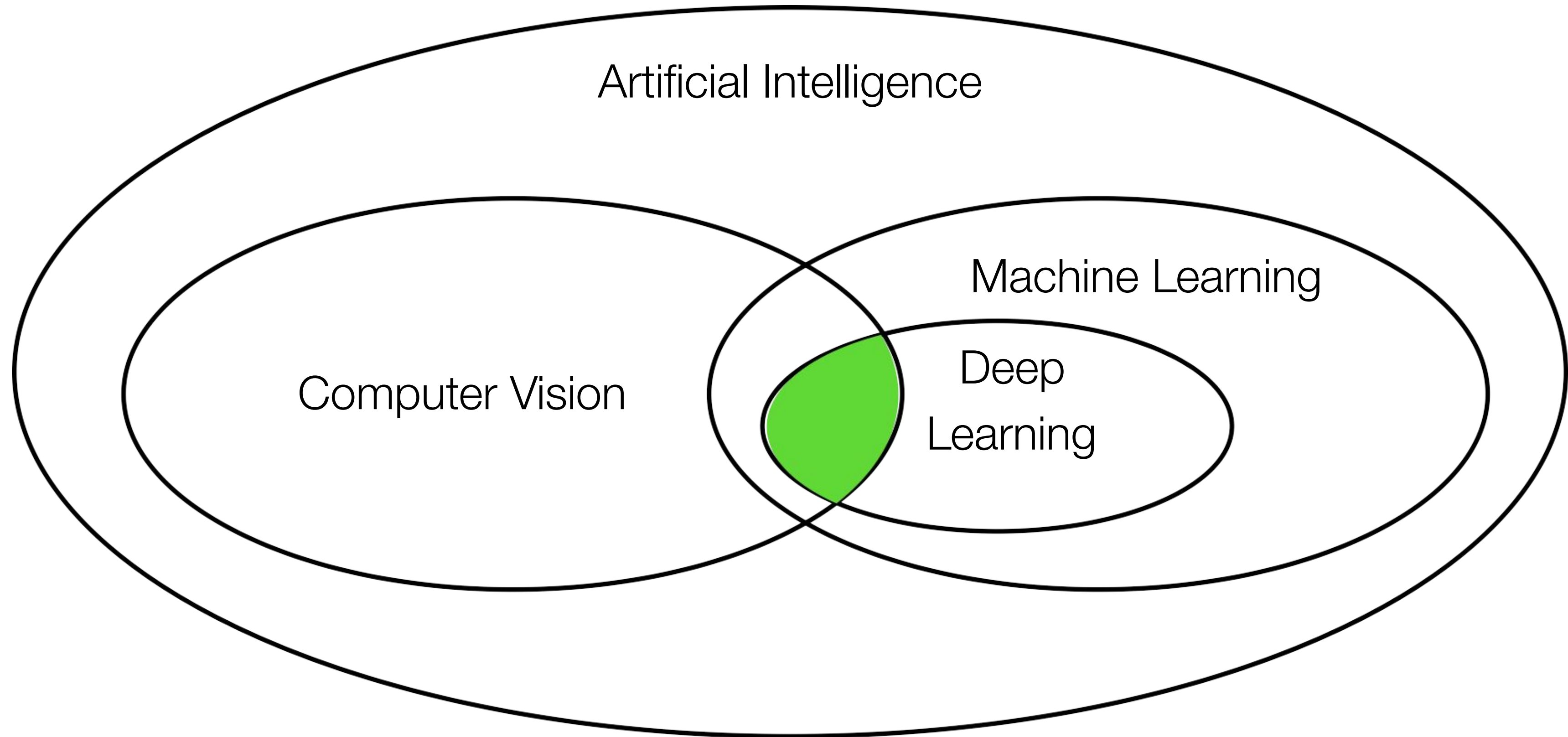
With the rapidly growing volume of data, manual analysis is no longer feasible, making automatic methods essential.

Artificial Intelligence



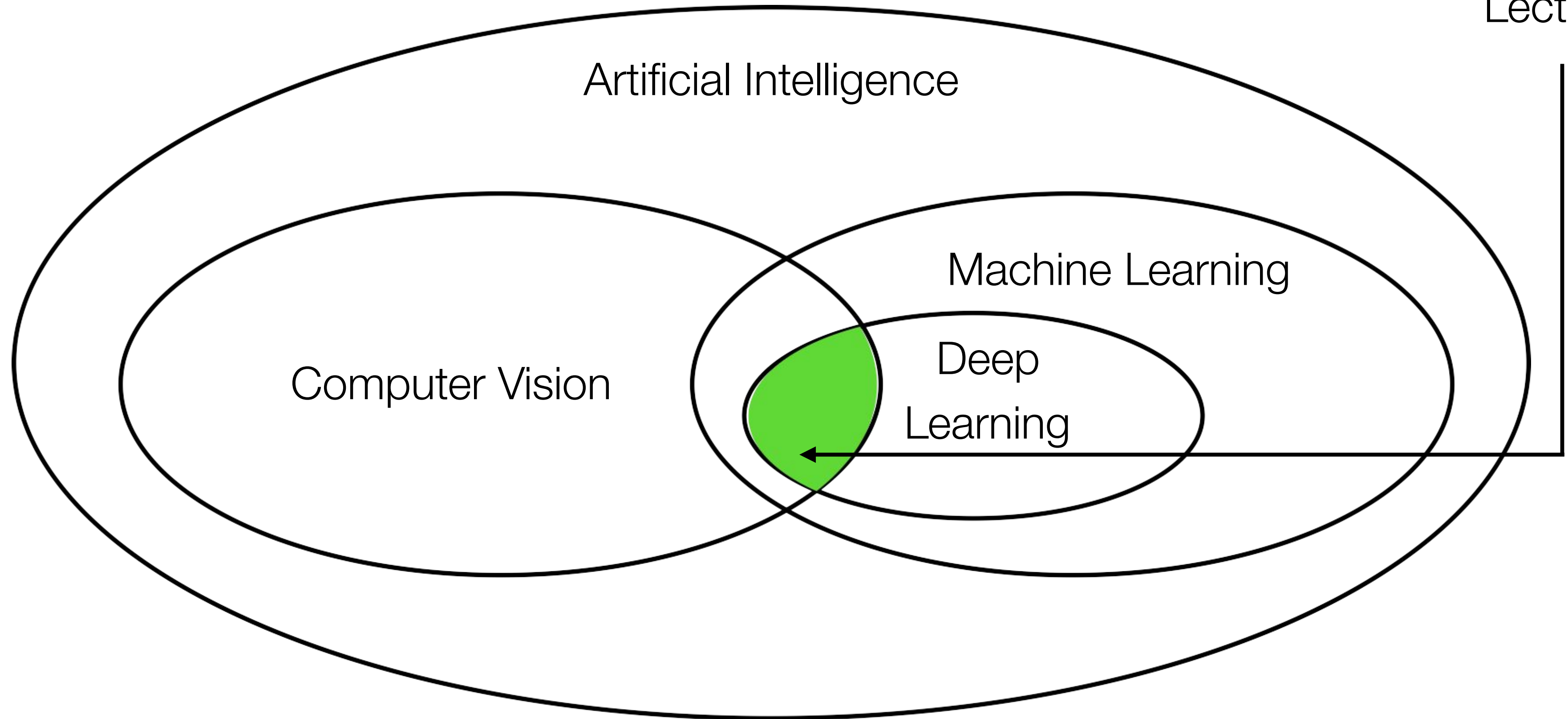
The ability of a computer or other machine to perform those activities that are normally thought to require intelligence (learning from data, decision making ...)

Deep Learning for Computer Vision



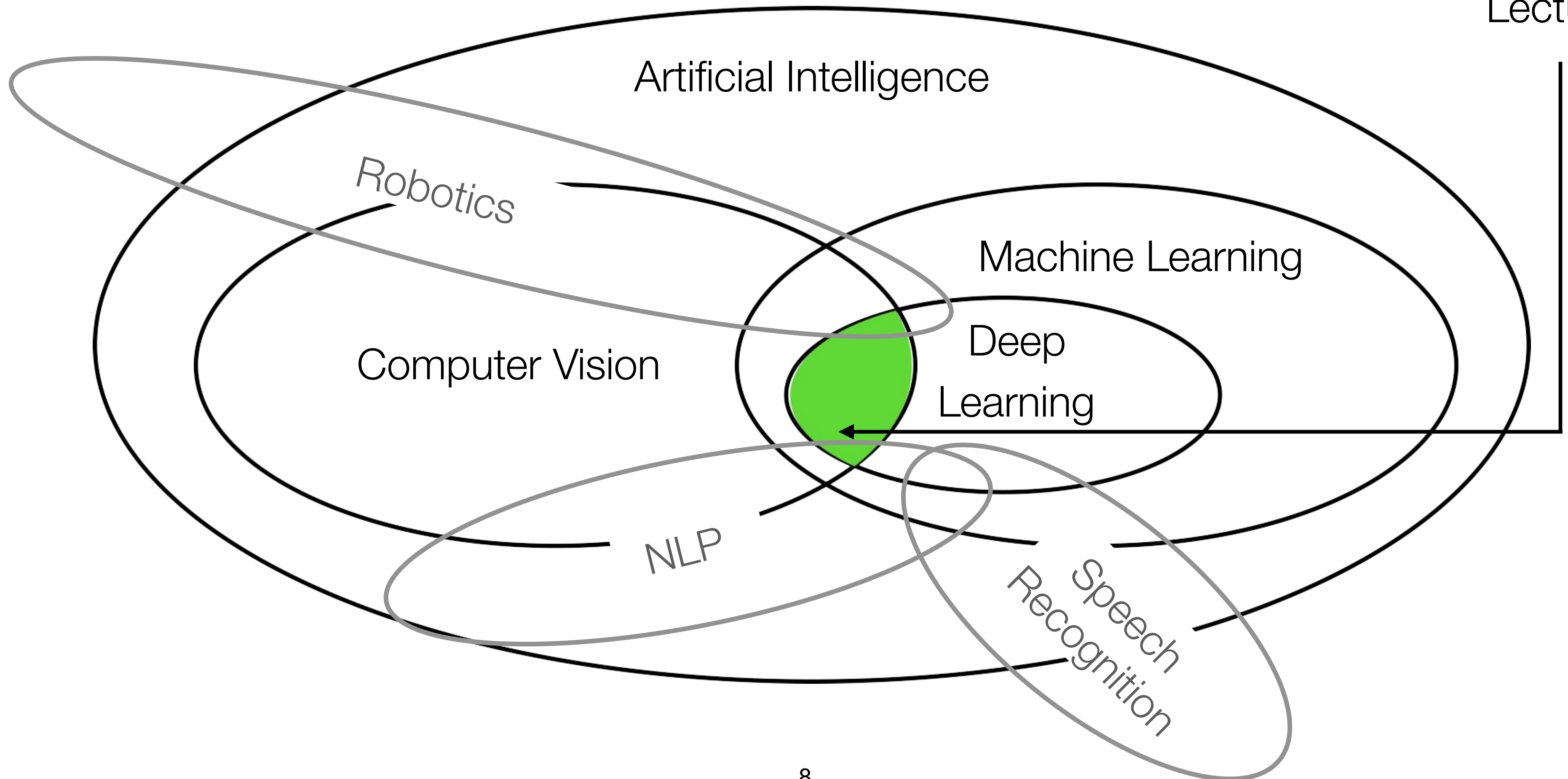
Deep Learning for Computer Vision

This
Lecture

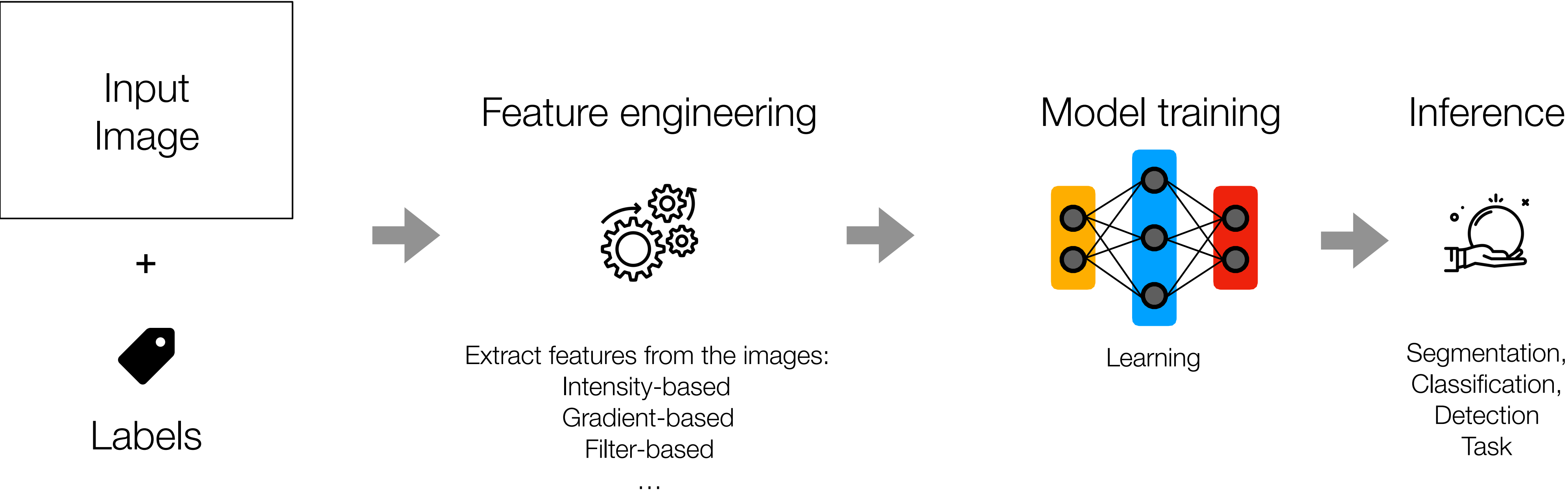


Deep Learning for Computer Vision

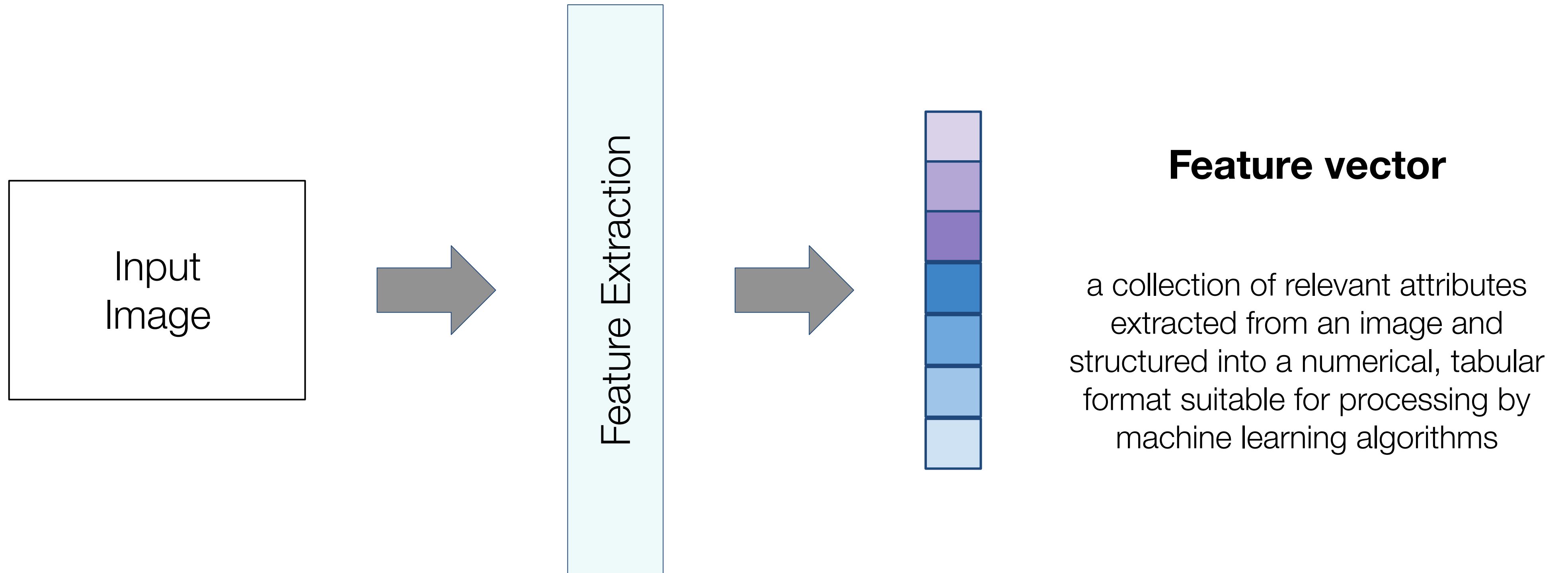
This
Lecture



Machine Learning



Handcrafted feature extraction



Handcrafted feature extraction

Pros:

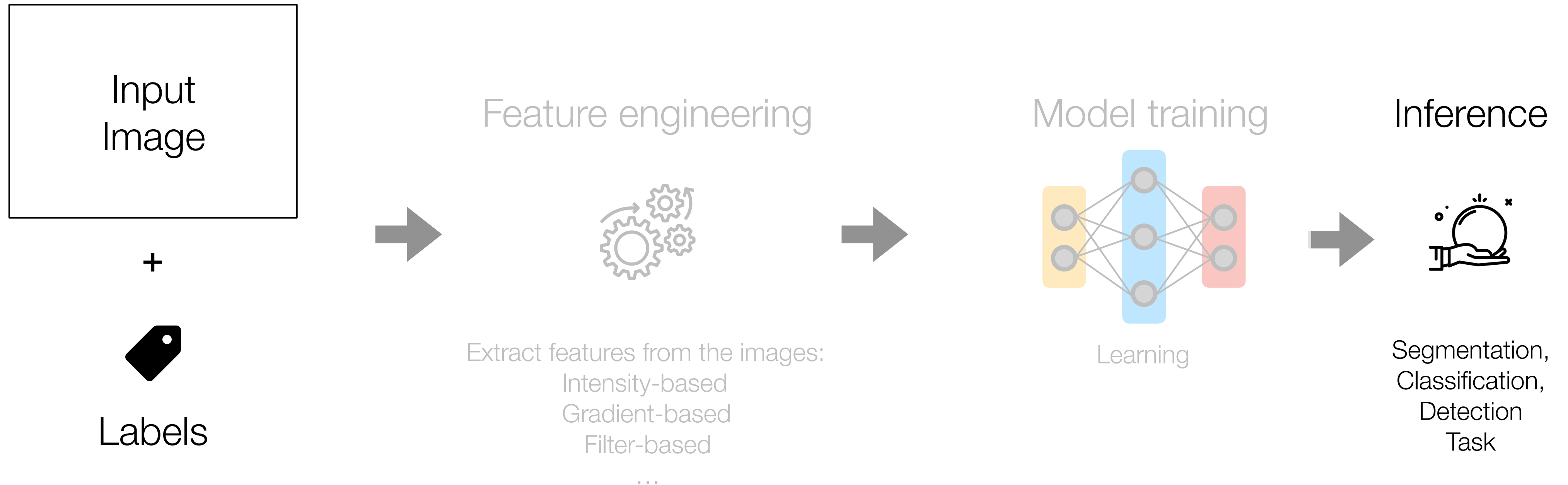
- Exploit domain information / a priori information
- Features extracted are interpretable (you actually know what is driving classification)
- Limited amount of data needed

Cons:

- Require a lot of design/programming effort
- Hard to manually capture subtle visual patterns that humans recognize effortlessly, making handcrafted features unreliable or impractical for many vision tasks.
- Risk of overfitting to the training set used in feature design
- Not very general

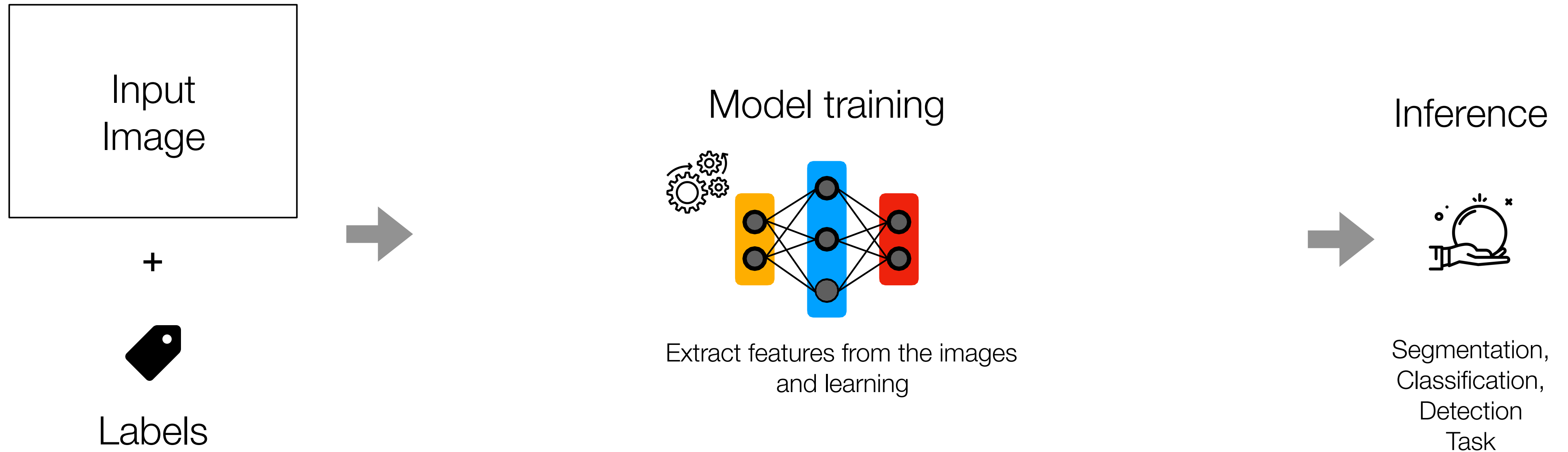
Deep Learning for Computer Vision

Deep Learning is an evolution of the traditional Machine Learning approach:



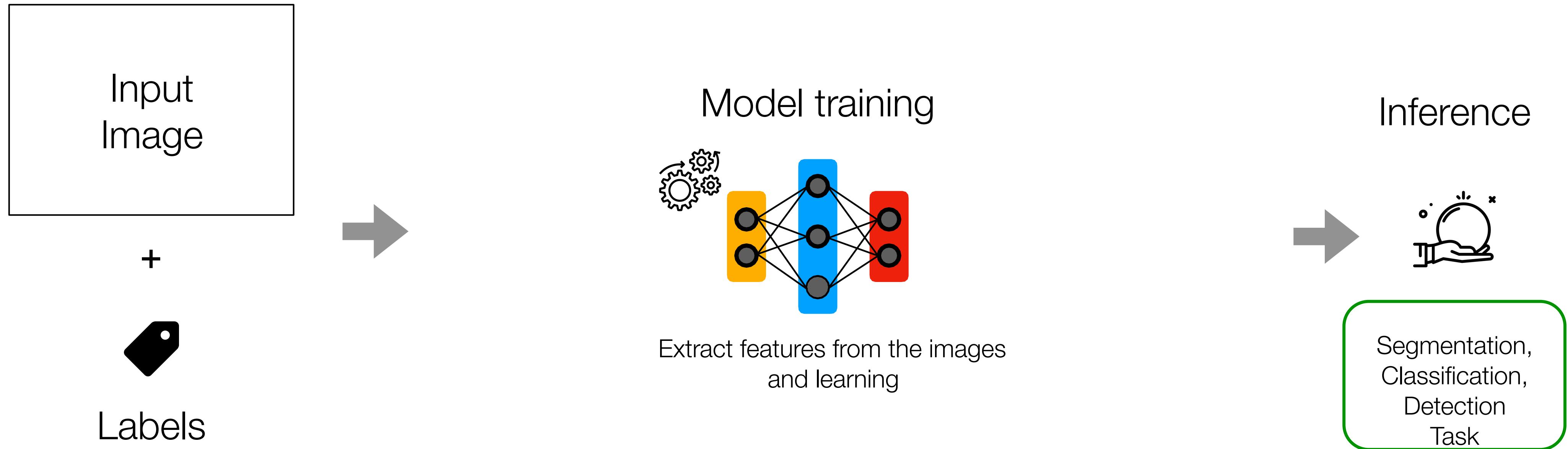
Deep Learning for Computer Vision

Deep Learning is an evolution of the traditional Machine Learning approach:



Deep Learning for Computer Vision

Deep Learning is an evolution of the traditional Machine Learning approach:

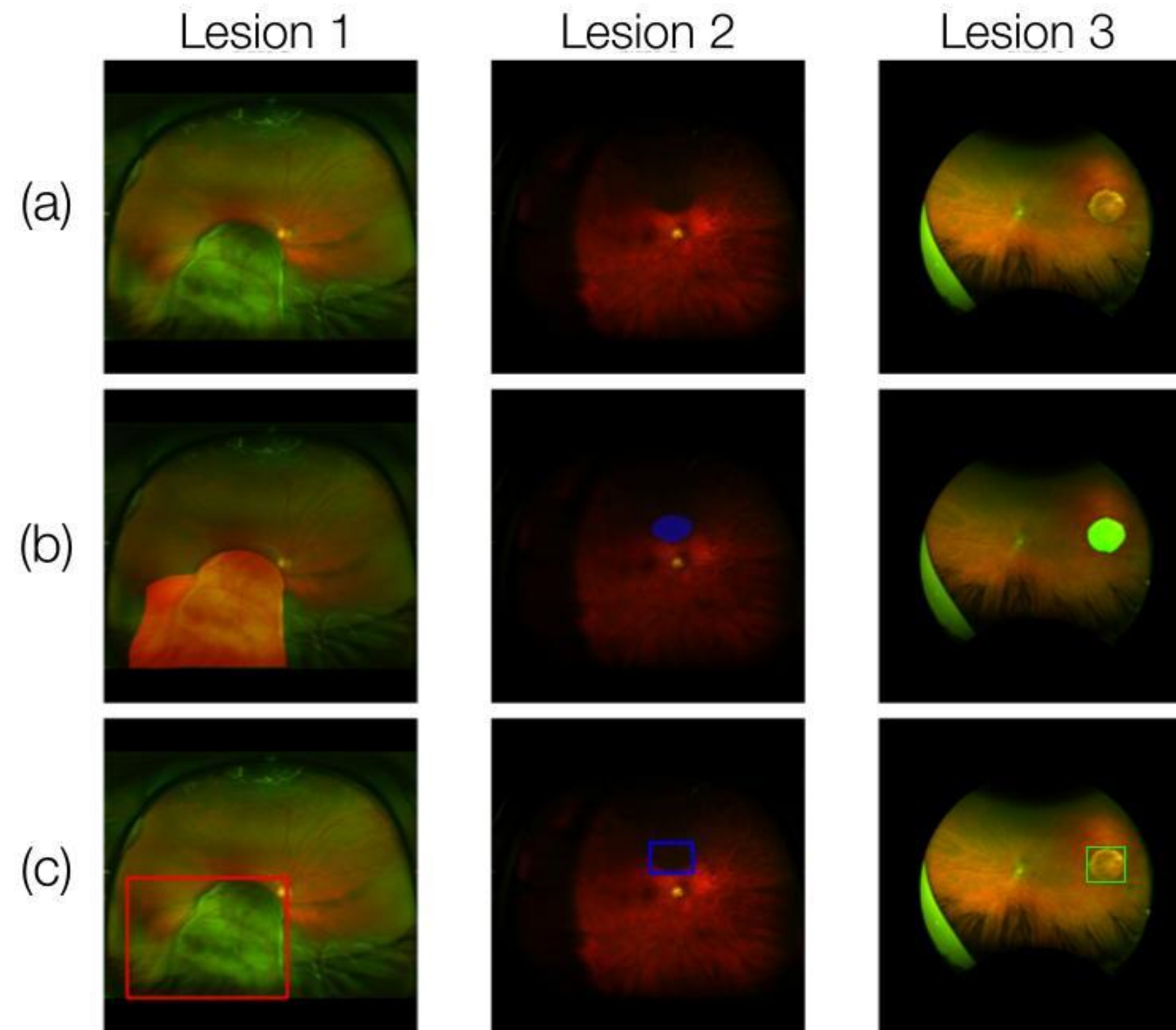


Common Tasks Done for Image Analysis

We can benefit from automated image analysis for 3 main purposes:

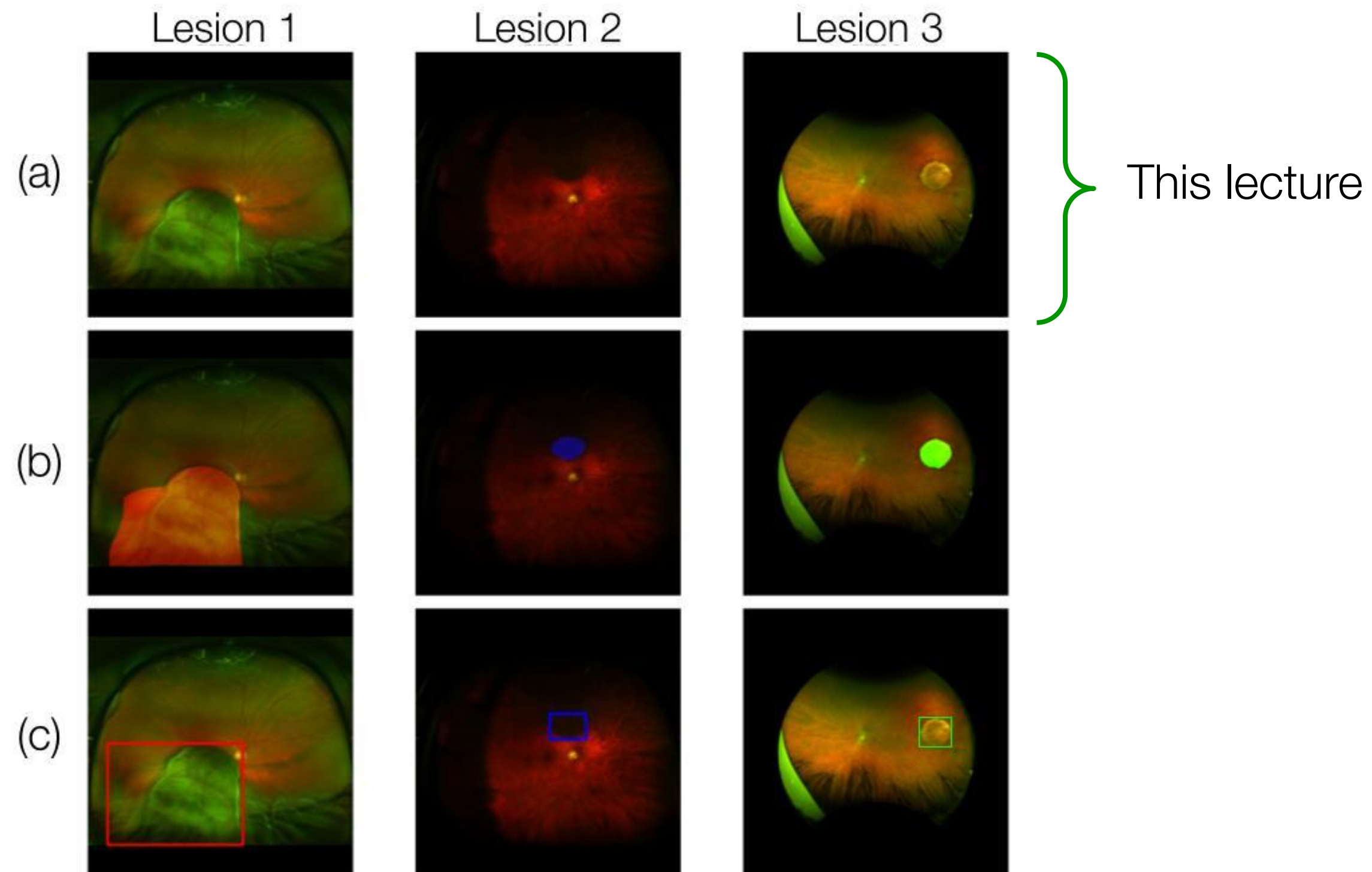
- 
- Increased complexity
- Classification: provide a label at the image level; e.g.: whether an image contains a tumor or not
 - Detection: locate the class of interest in the image; e.g. not only say if the X-Ray contains a tumor or not, but also draw a box (or a circle, or something else) around it.
 -
 - Segmentation: precisely annotate at the pixel-level the class of interest; e.g. put to 1 all the pixels belonging to the tumor, leave to 0 everything else.

Common Tasks Done for Image Analysis



This image shows three different types of lesions. In each column: (a) is the input image with its **image-level label** indicating the lesion type; (b) is the **segmentation mask** that highlights the pixels belonging to the lesion; (c) shows **bounding boxes** that localize the lesion within the image

Common Tasks Done for Image Analysis



This image shows three different types of lesions. In each column: (a) is the input image with its **image-level label** indicating the lesion type; (b) is the **segmentation mask** that highlights the pixels belonging to the lesion; (c) shows **bounding boxes** that localize the lesion within the image

Data-driven features

Features are directly learned from data during model training

- No manual design required: the model discovers the most relevant visual patterns on its own.
- Capture subtle, high-level representations (e.g., shapes, textures, objects, context) that are hard to hand-engineer.
- Learned features are optimized for the specific task

Deep Learning Theoretical Foundations

At the core of Deep Learning are **artificial neurons**, inspired by the way biological neurons process information in the brain

- In the field of AI, research has been inspired a lot by concepts from neuroscience.
- In the human brain, computations are performed with small **non linear** units (neurons), which work **in parallel** and they are **redundant** and thus fault tolerant
- The human brain has a lot of computing units: 10^{11} neurons, and each neuron is connected and exchange informations with other neurons.

Our brain is a **distributed and parallel computing machine**

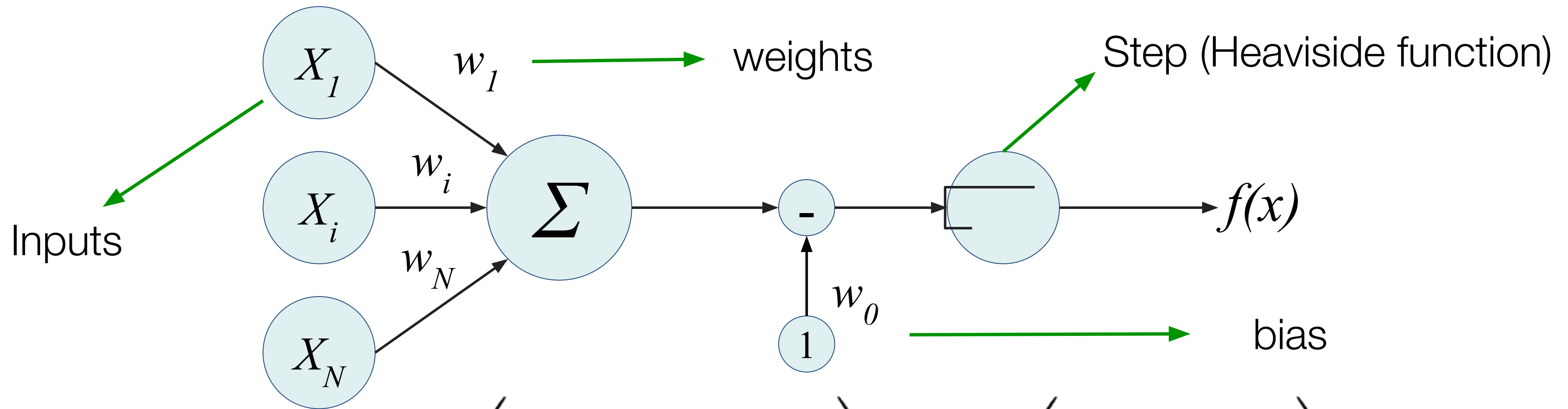
Idea: develop a computational model based on the brain!

Computation in Biological Neurons

- A biological neuron is a cell that receives some charge from other neurons, through dendrites. This charge accumulates in the neuron and at a certain point, a **threshold** is overpassed, and the accumulated charge is **released** and reaches other neurons, through synapses
- Information is transmitted through chemical mechanisms

The Artificial Neuron

To reproduce this model, the **perceptron** is proposed. This is a model of a single neuron and the simplest form of a neural network



$$h_j(x \mid \mathbf{w}, b) = h_j \left(\sum_{i=1}^I w_i \cdot x_i - b \right) = h_j \left(\sum_{i=0}^I w_i \cdot x_i \right) = h_j(\mathbf{w}^T \mathbf{x})$$

From the Perceptron to Neural Networks

To solve non-linear problems, and represent more complex functions, the perceptron is not sufficient.

- Allows only linear separation;
- The step activation function is not differentiable;

Therefore, from the perceptron, we go towards Multilayer Perceptrons (MLPs)

- Stack multiple layers of perceptrons (neurons), including one or more hidden layers;
- Use nonlinear and differentiable activation functions to enable gradient-based optimization techniques, which are critical for effectively training neural networks;

We then go to Deep Neural Networks by stacking **many** layers

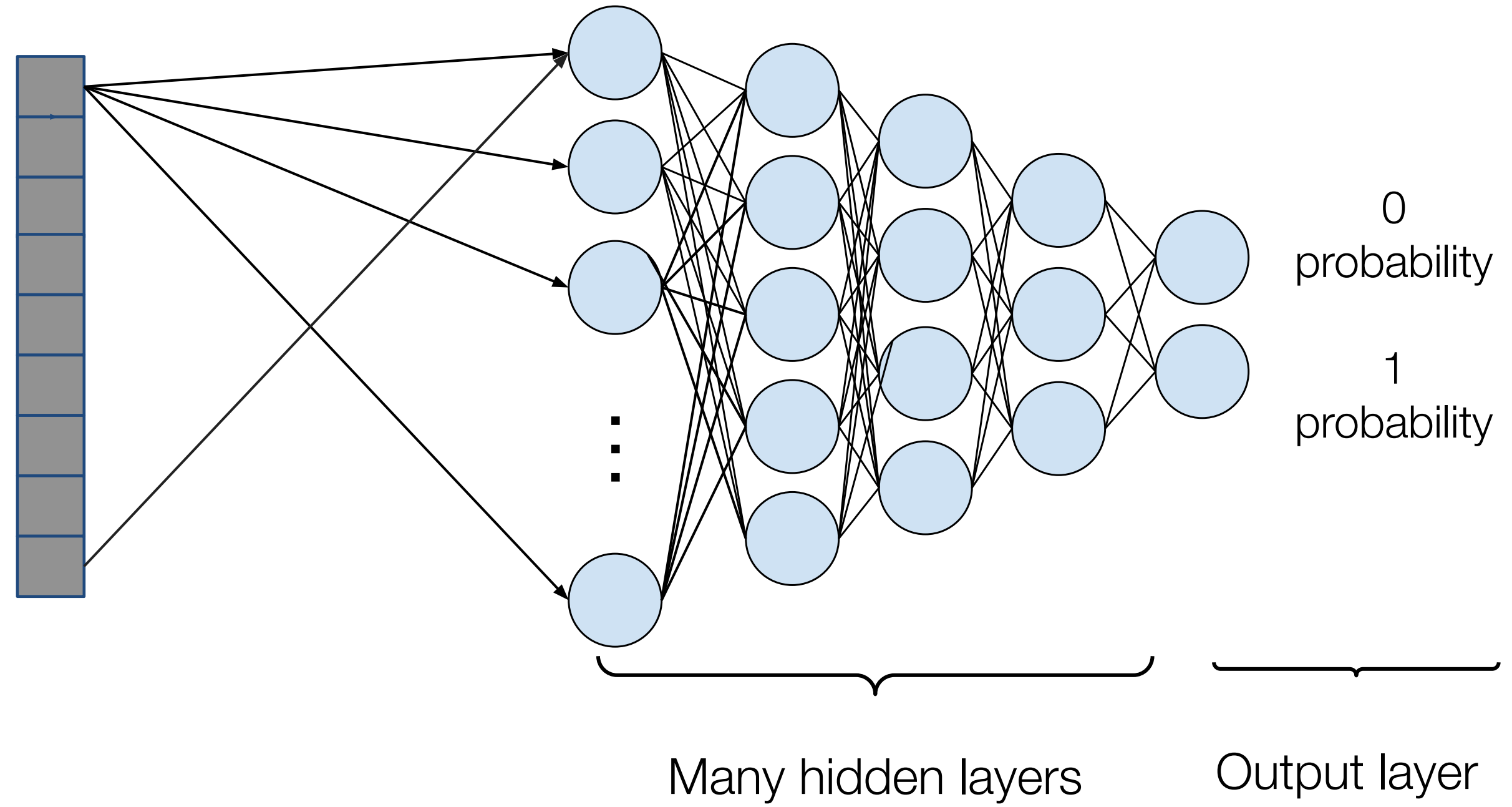
Deep Learning for Computer Vision

Building artificial systems
that **learn** from data and experience

Deep Learning for Computer Vision

Building artificial systems
that **learn** from data and experience
with
hierarchical learning algorithms made by **many “layers”**,
inspired by the brain

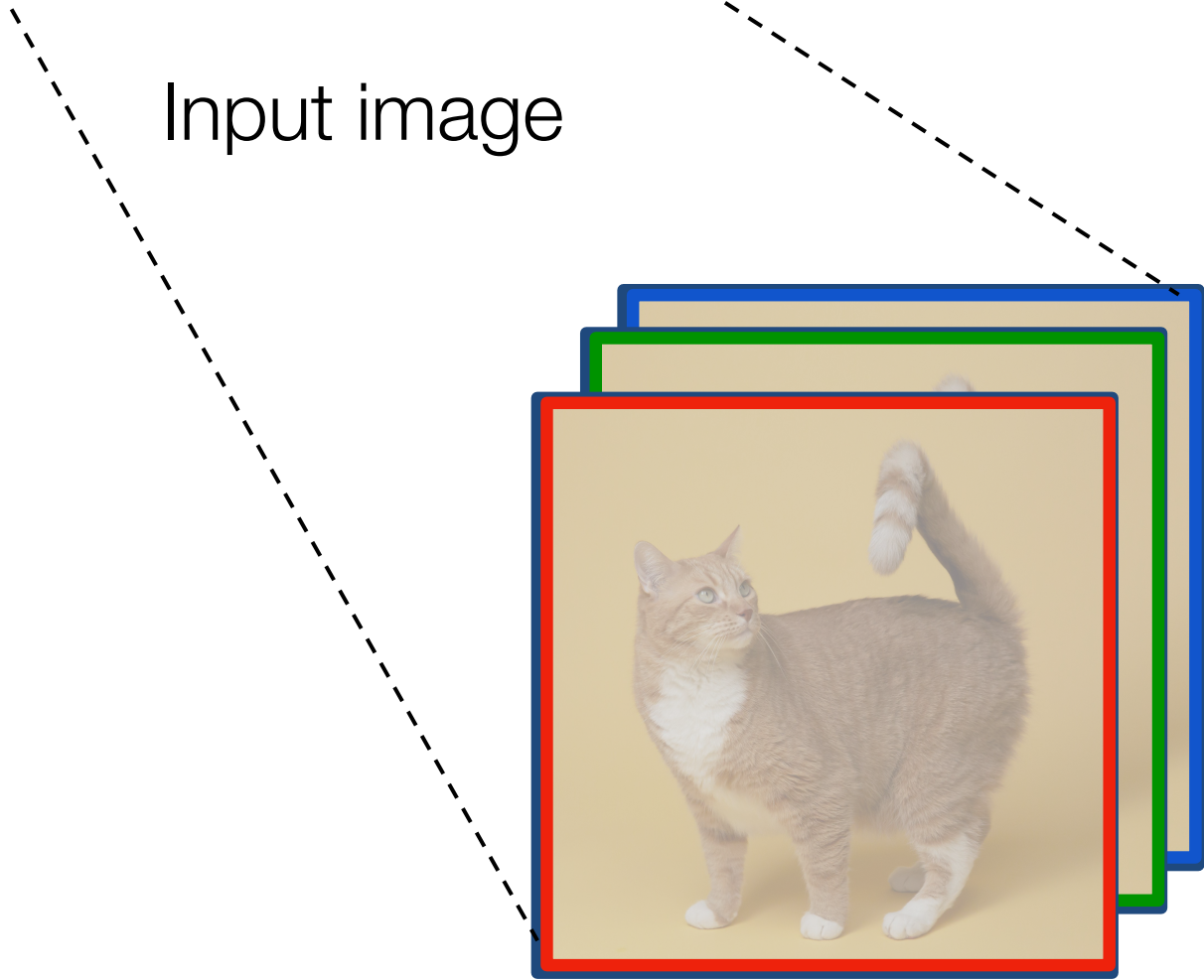
Deep Neural Networks



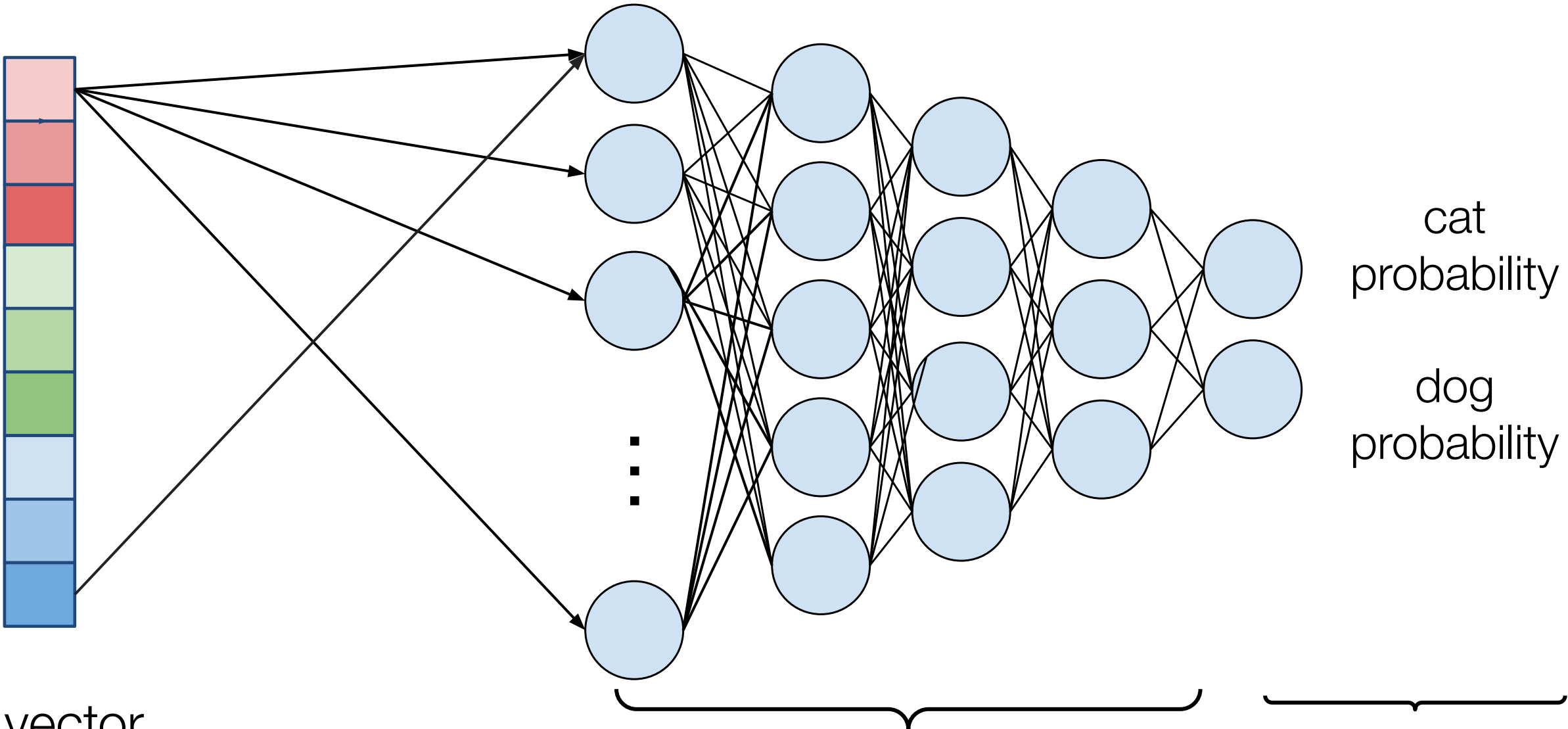
Deep Neural Networks



1D vector
(unroll the image
column-wise)



RGB channels



Many hidden layers

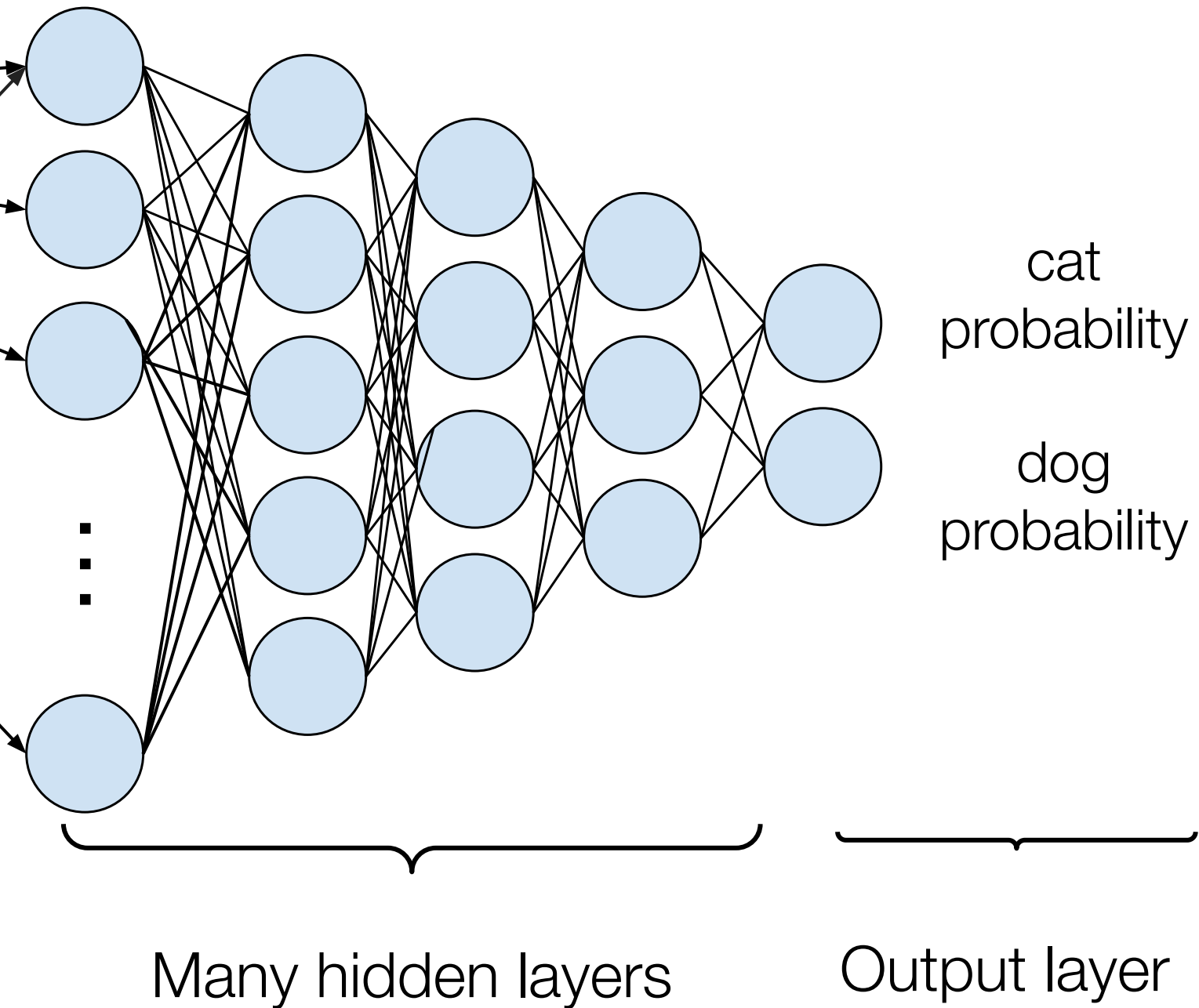
Output layer

Deep Neural Networks

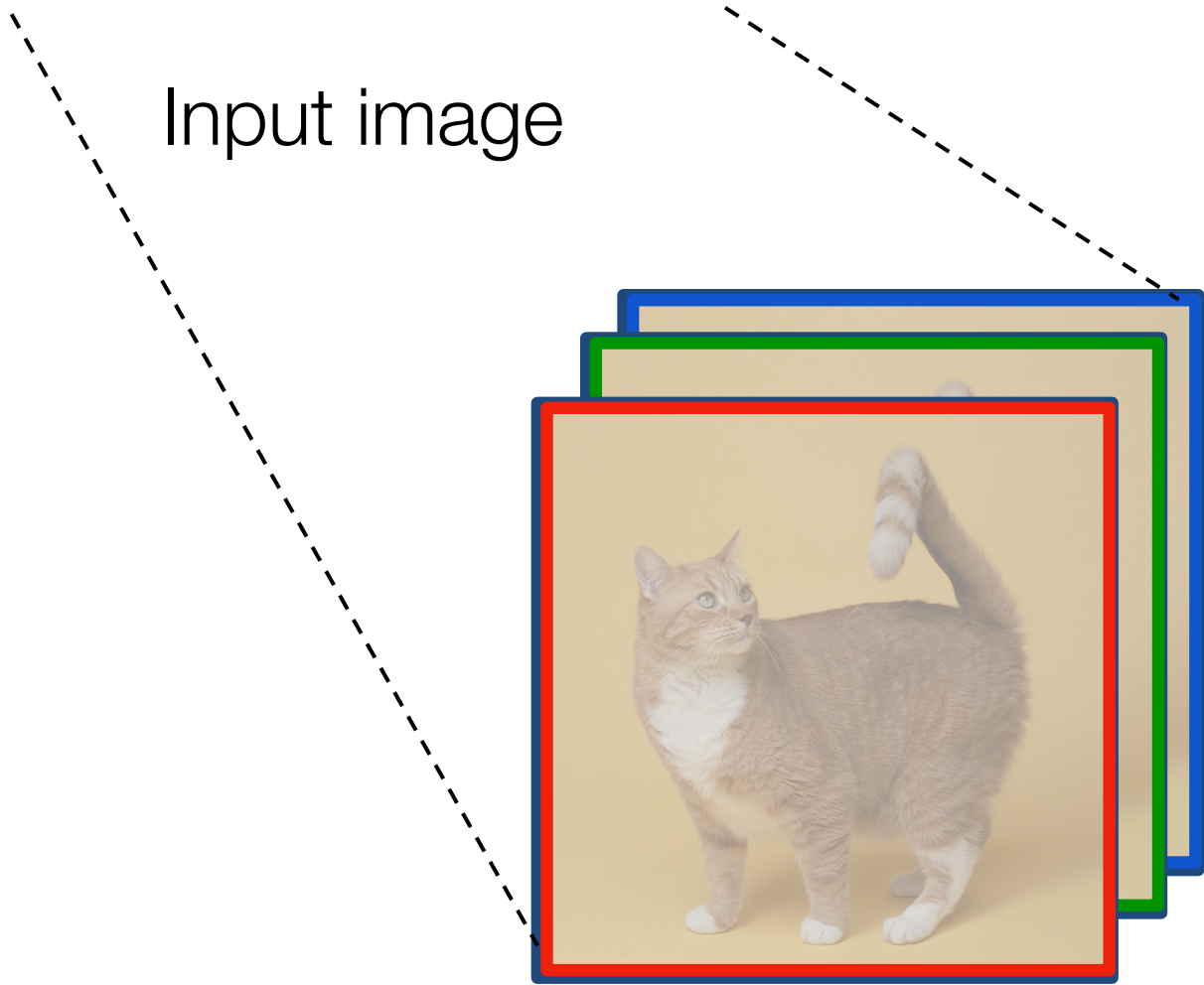


1D vector
(unroll the image
column-wise)

$224 \times 224 \times 3 = 150528$
input neurons!



We can see the FFNN as mapping
the unfolded input image to the class
with the highest probability



RGB channels

Problems with traditional Neural Networks

Images are not suitable for traditional neural networks;

- Traditional neural networks treat input data as flat vectors
 - too many parameters even in moderately sized images(e.g., $64 \times 64 \times 3 = 12288$ input features);
 - computationally expensive training;
 - large memory requirements;
 - Ignoring spatial structure: flattening images removes the spatial information like local patterns, losing information about edges, textures and shapes;
 - Lack of parameter reuse; separate weights are learned for every input feature.

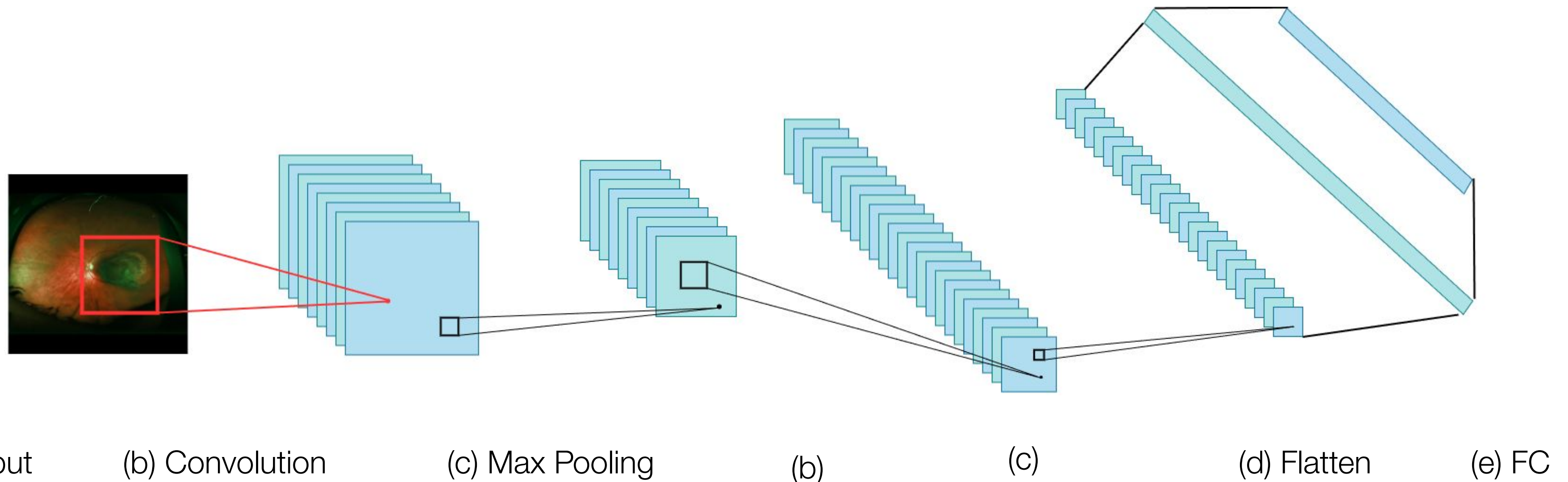
Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are designed to process data with a grid-like structure, such as images. They use the mathematical operation of convolution, where small filters (kernels) slide over the image to extract local patterns. Each filter produces a feature map, highlighting where specific patterns occur in the image. As we go deeper into the network, progressively lower-level features are learned: early layers learn simple features (e.g., edges), while deeper layer extract semantic concepts. This procedure allows CNNs to build a hierarchical understanding of images, similar to how the human visual system works.

- Sparse connectivity: each neuron in a convolutional layer looks only at a small part of the image (e.g., a 3x3 or 5x5 area);
- Weight sharing (parameter reuse): the same set of filter weights is applied across all spatial locations in the image;
- Preserving spatial relationships: CNN's keep the matrix nature of images; pooling layers summarize local details, and introduce translation invariance;

Convolutional Neural Network - A General Architecture

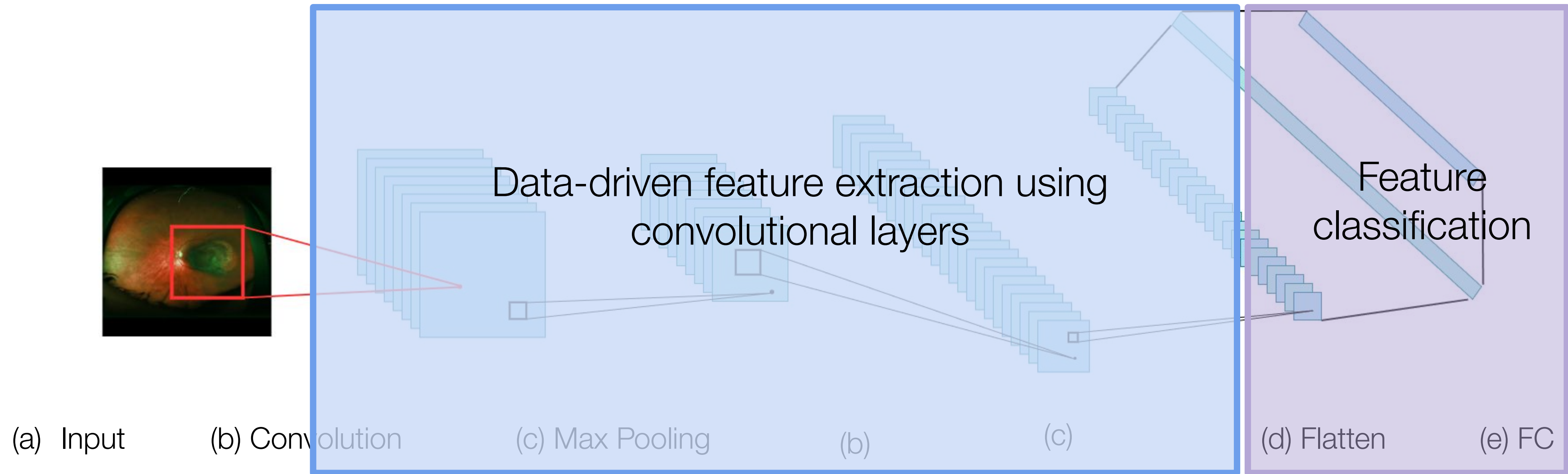
Classic architecture: [Conv, ReLU, Pool] x N, flatten, [FC, ReLU] x N, FC



Convolutional Neural Network - A General Architecture

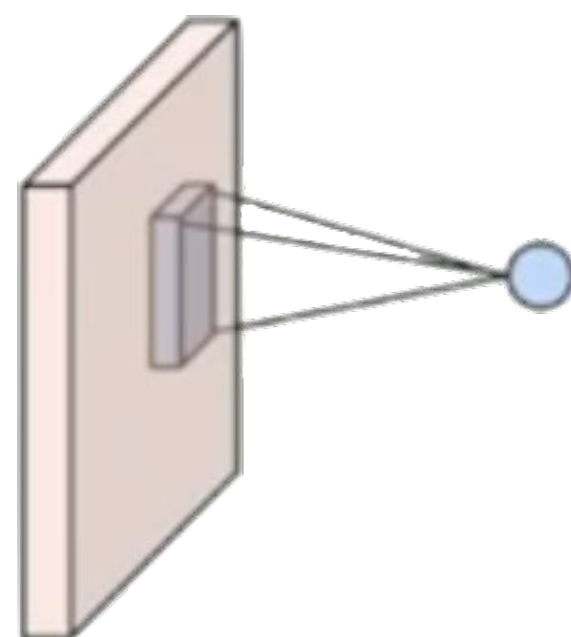
Classic architecture: [Conv, ReLU, Pool] x N, flatten, [FC, ReLU] x N, FC

Encoder - Feature extraction Fully connected neural network

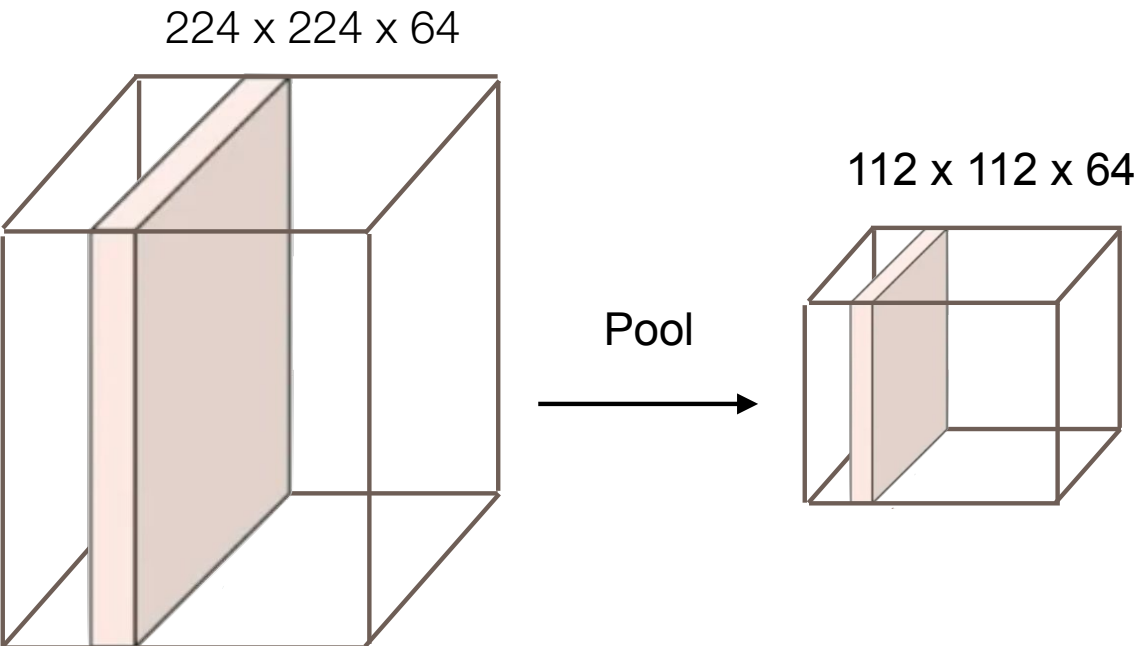


Convolutional Neural Network- Components

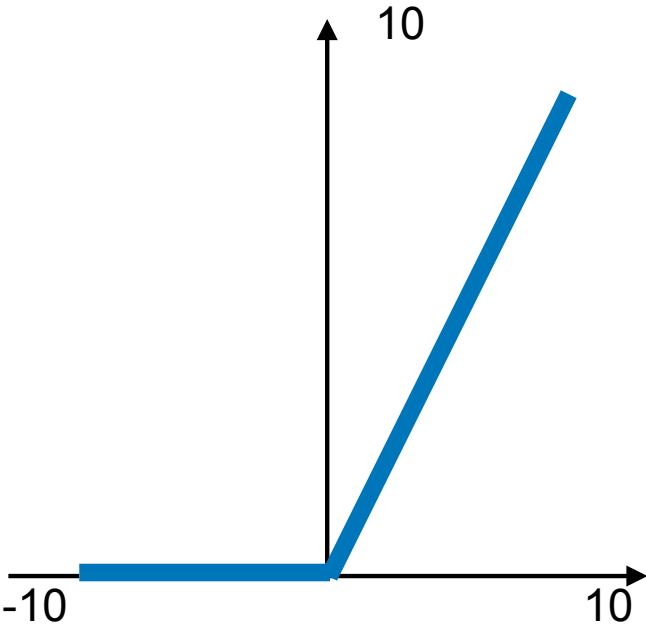
Convolution Layers



Pooling Layers



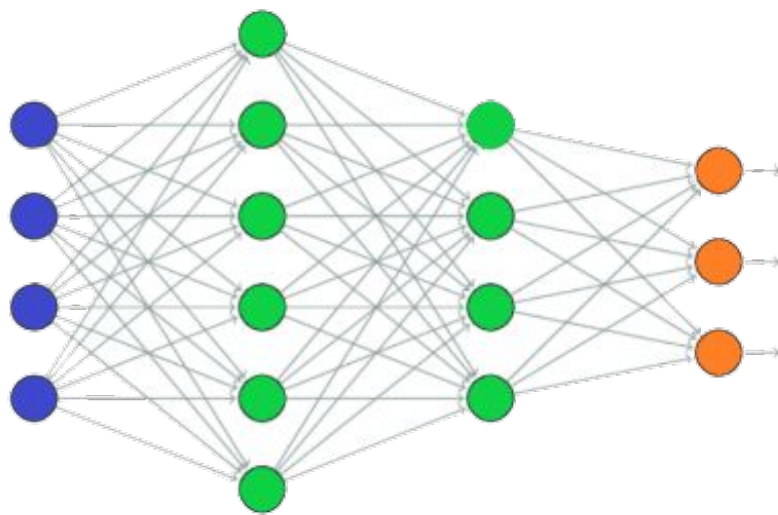
Activation Function



Normalization Layers

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

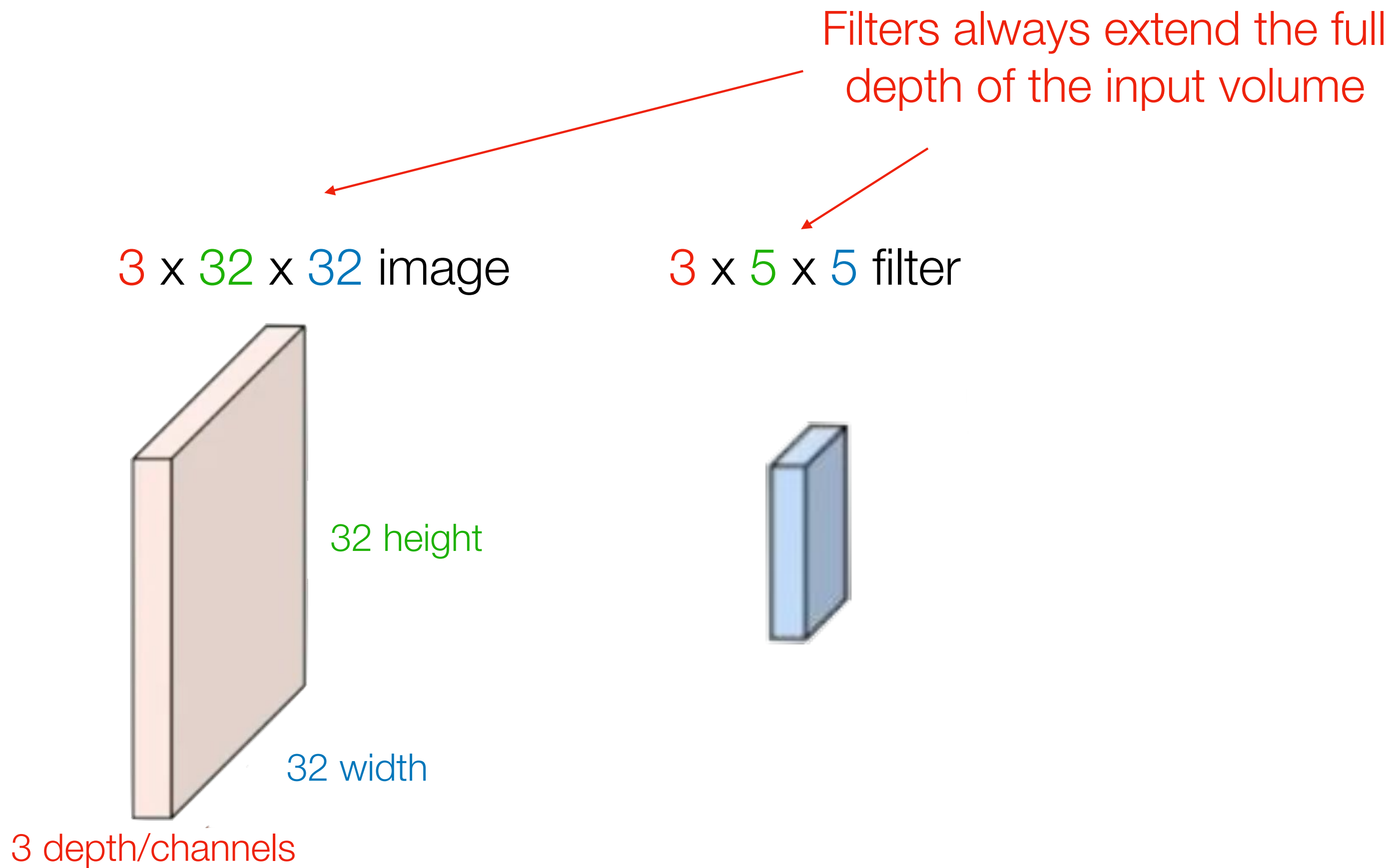
Fully-Connected Layers



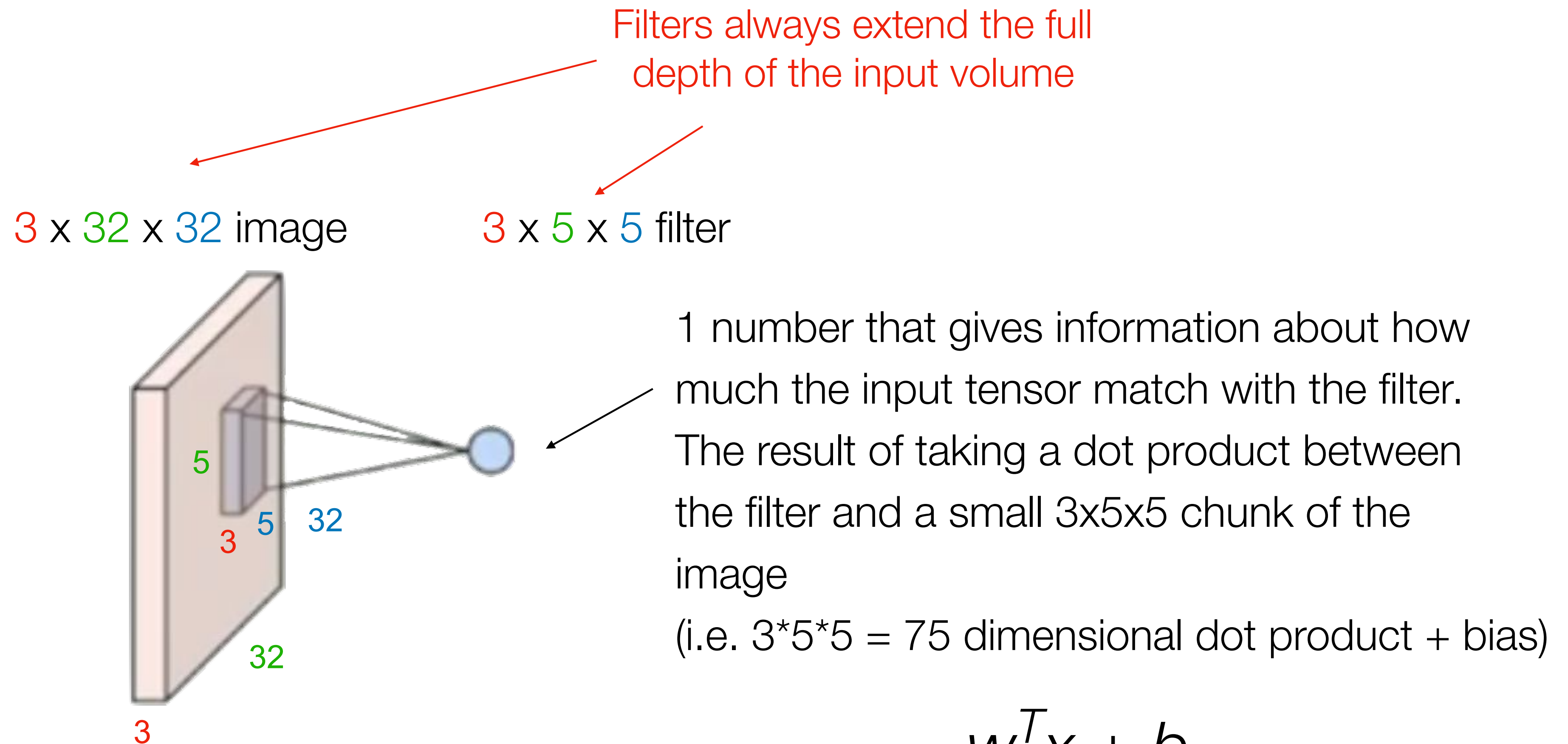
SoftMax Layers

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Convolution Layer

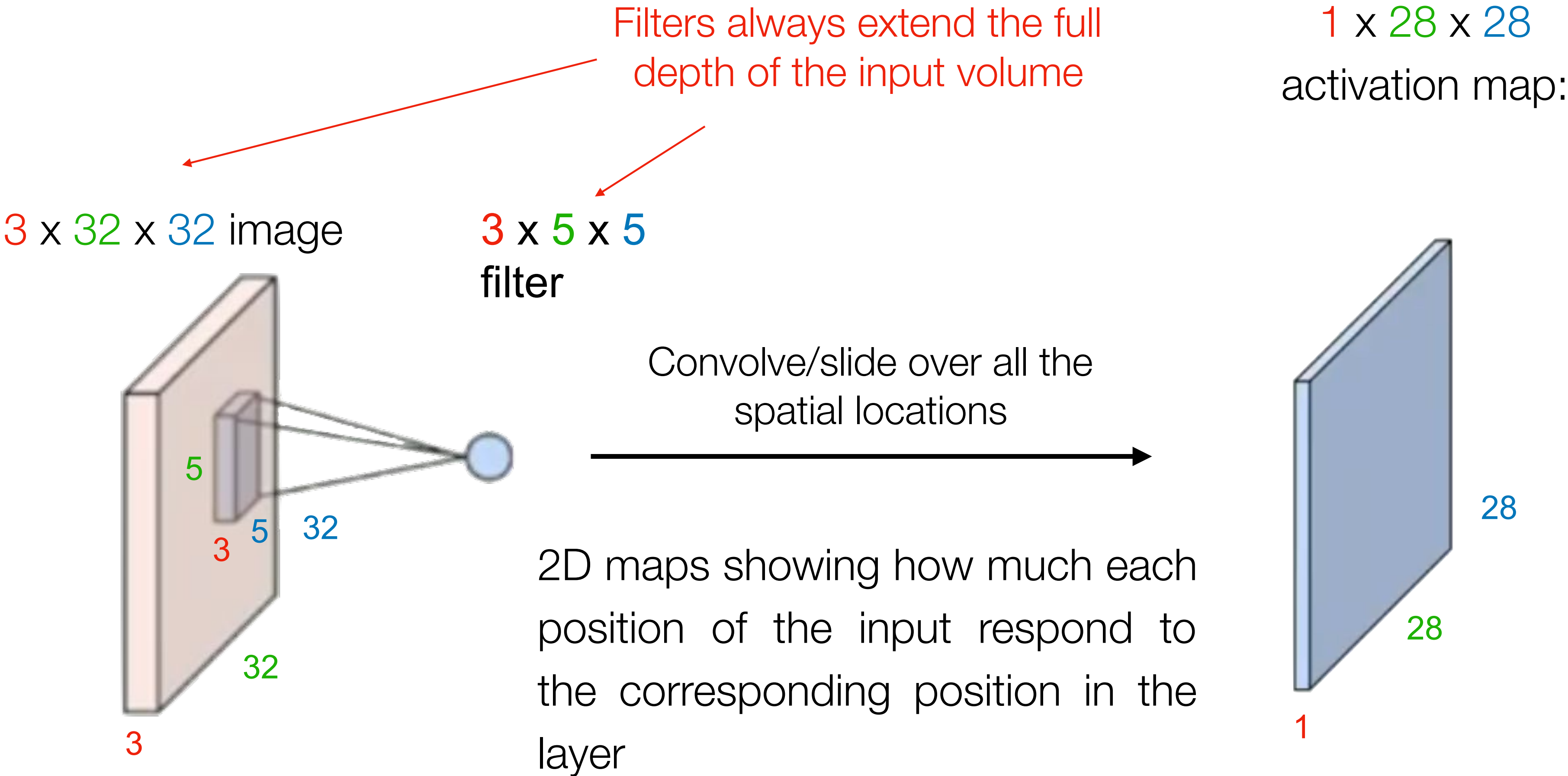


Convolution Layer



$$w^T x + b$$

Convolution Layer

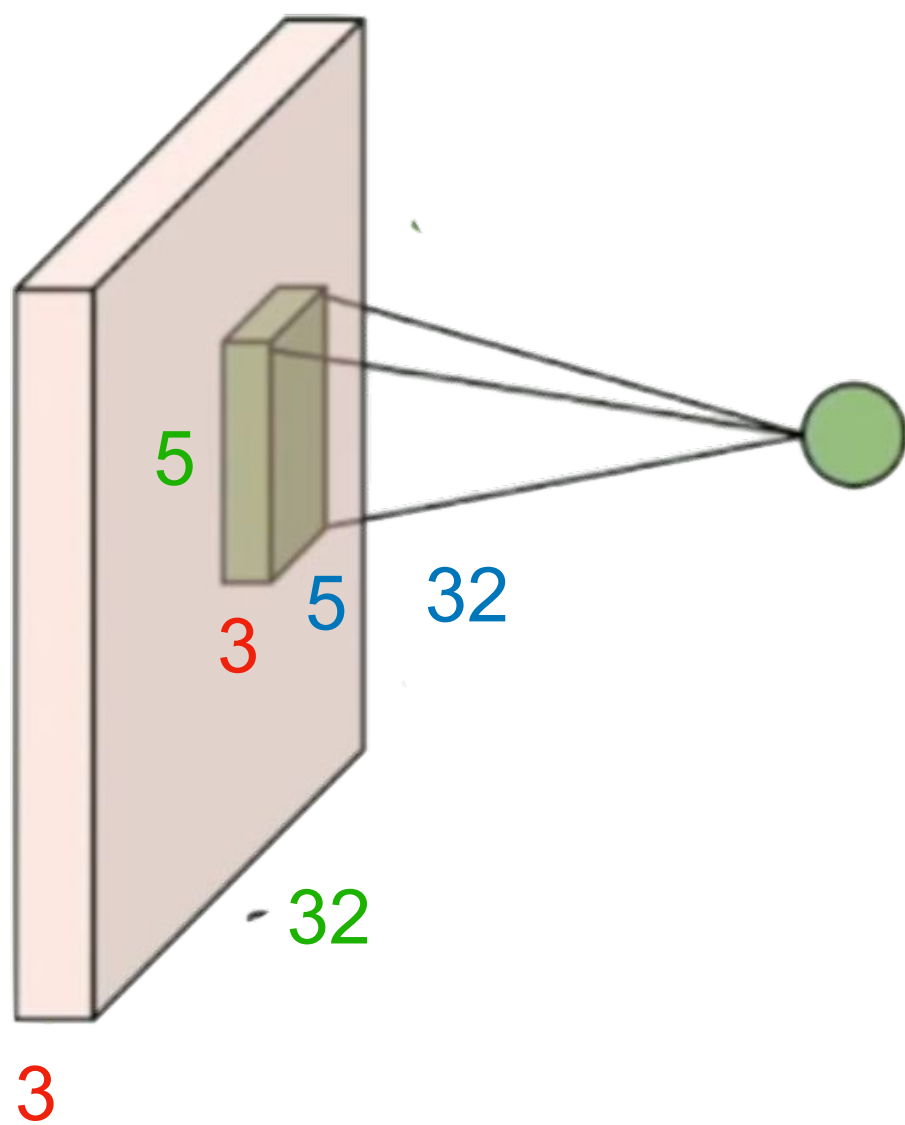


Convolution Layer

Consider repeating with
a second (green) filter:

3 x 32 x 32 image

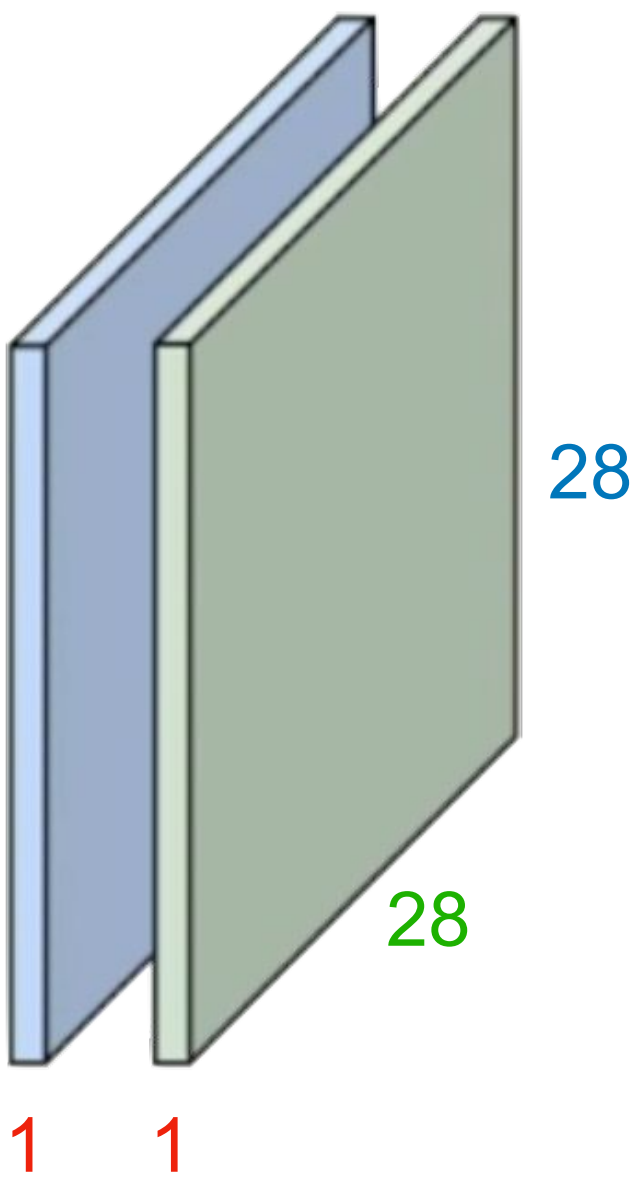
3 x 5 x 5 filter



Convolve/slide over all the
spatial locations

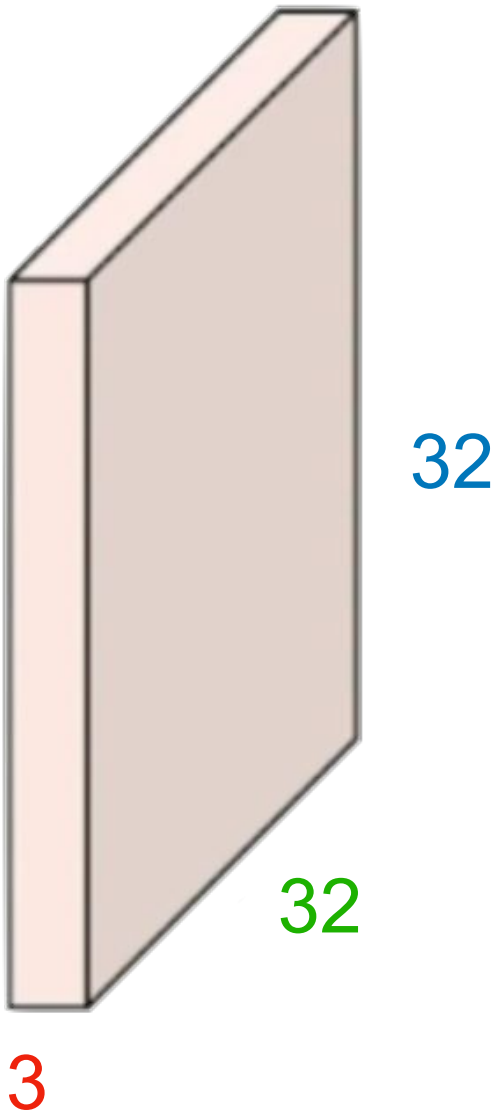


Two
1 x 28 x 28
activation maps



Convolution Layer

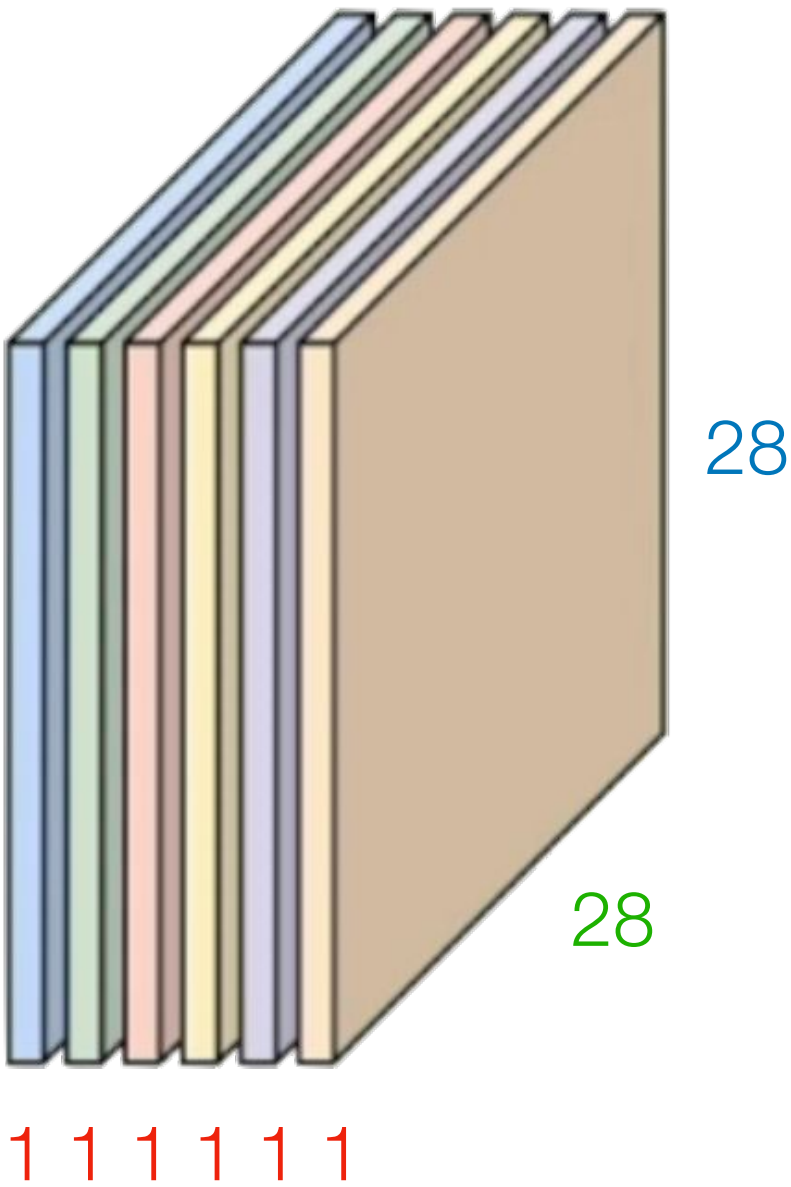
3 x 32 x 32 image



6-dim bias vector:



6
1 x 28 x 28
activation maps



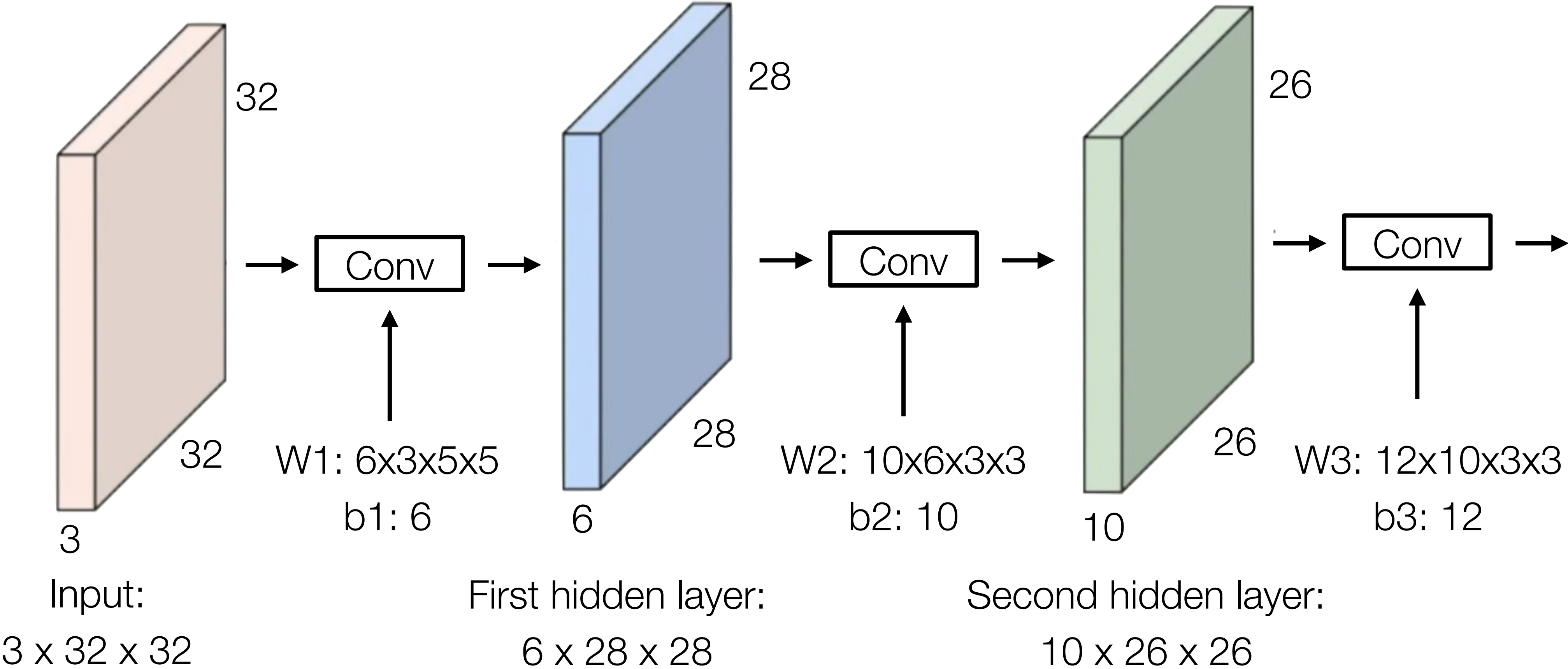
Convolution
Layer



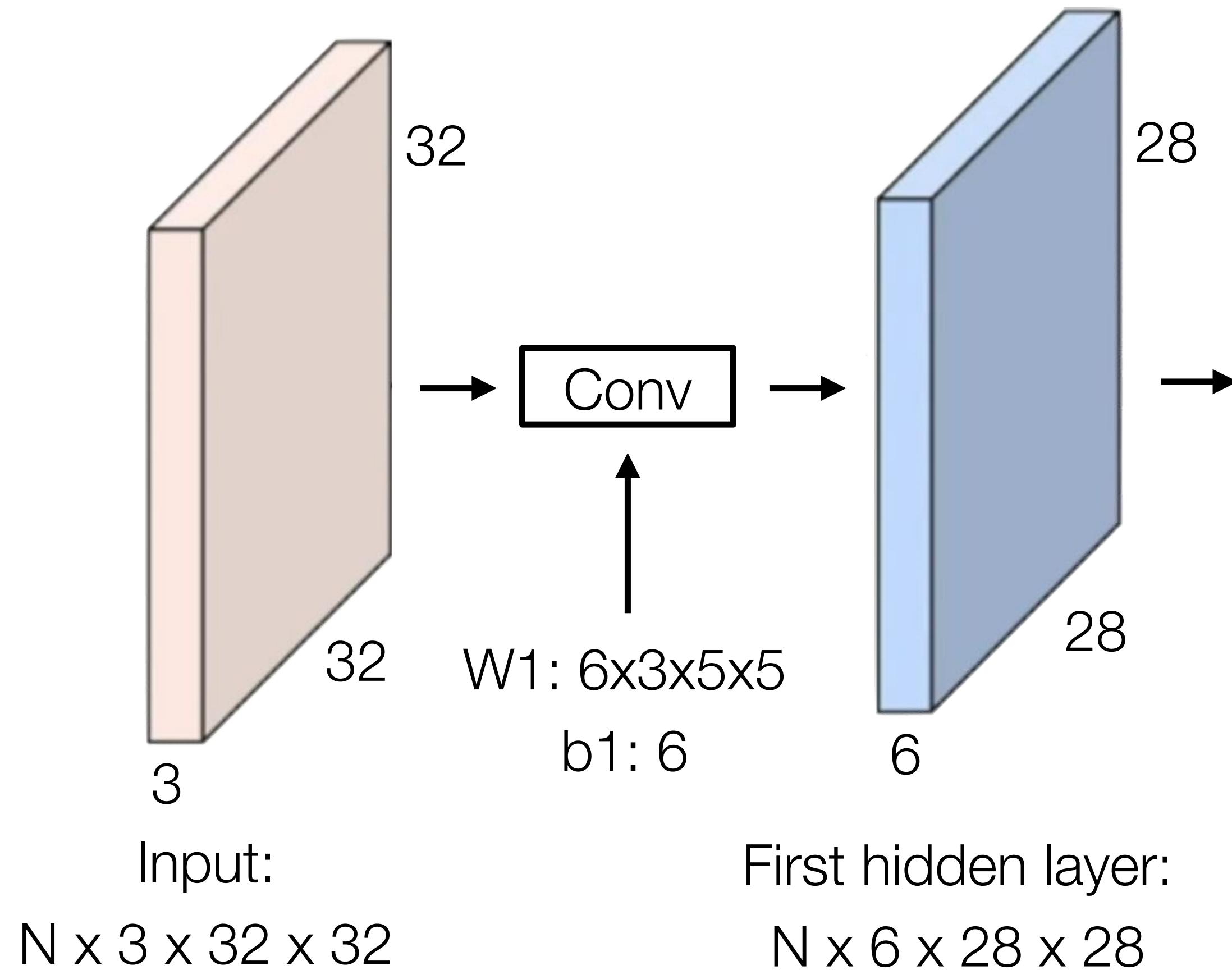
6 x 1 x 5 x 5
filters

Stacking Convolutions

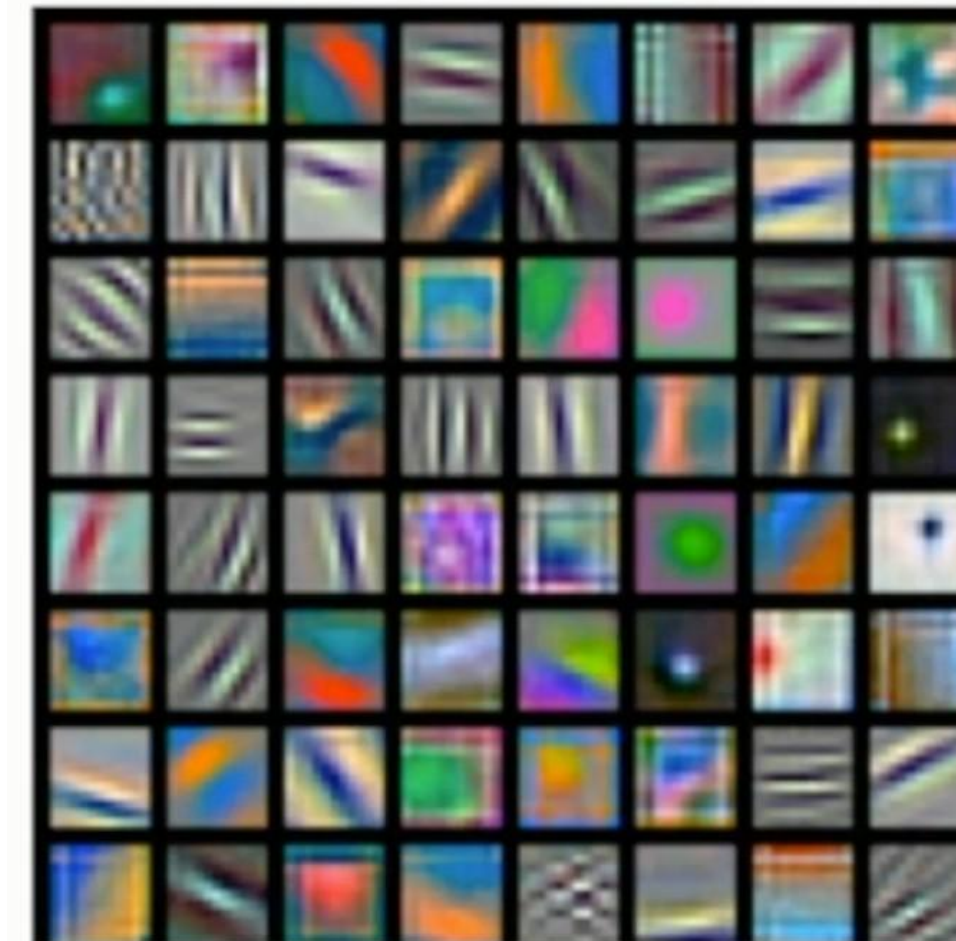
The number of output feature maps at each layer depends on the number of filters



What do convolutional filters learn?



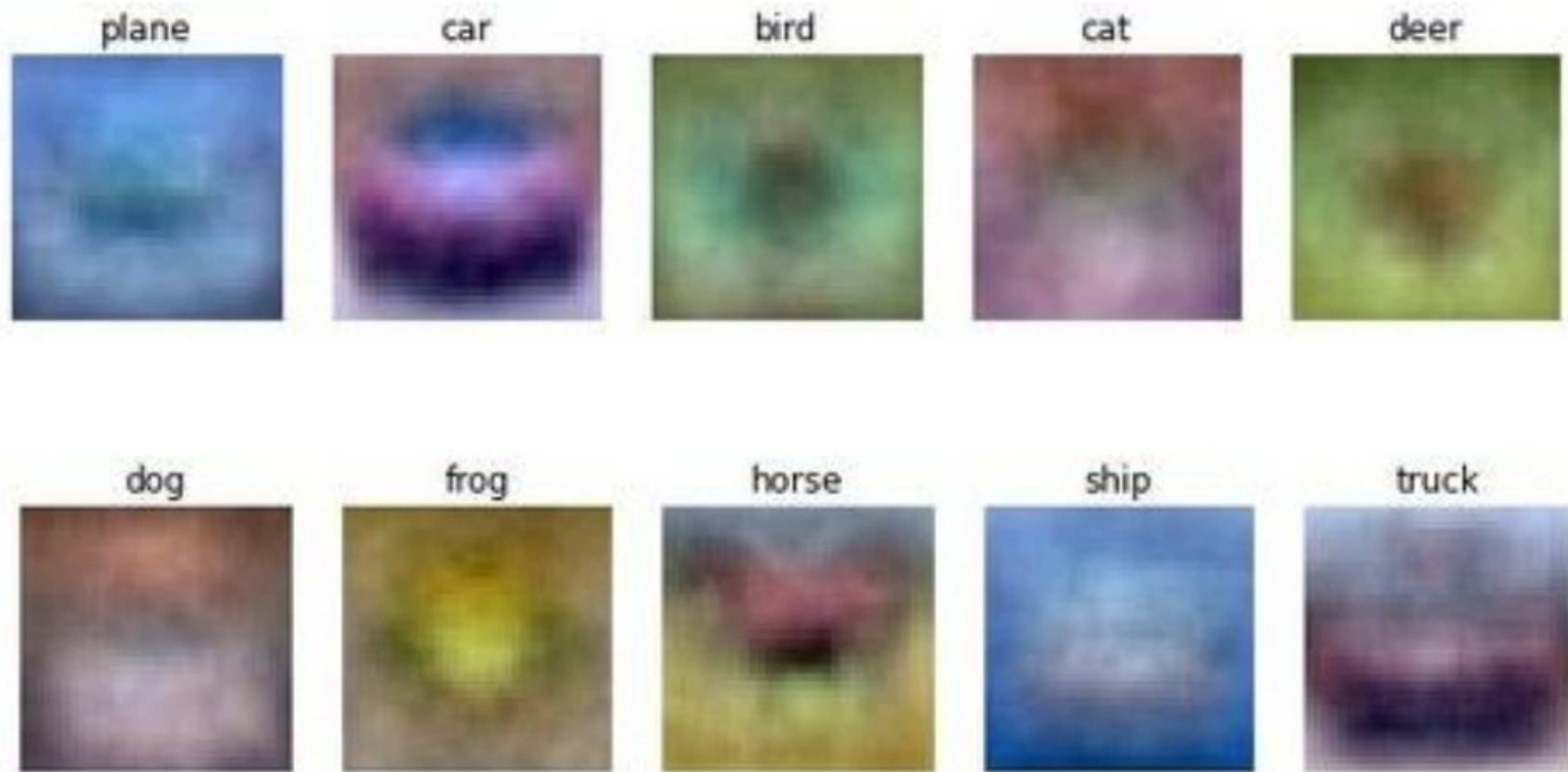
AlexNet: 64 filters,
each $3 \times 11 \times 11$



First layer conv filters:
local image
templates

often learns oriented edges
or opposing colors

What do convolutional filters learn?



Stacking Convolutions

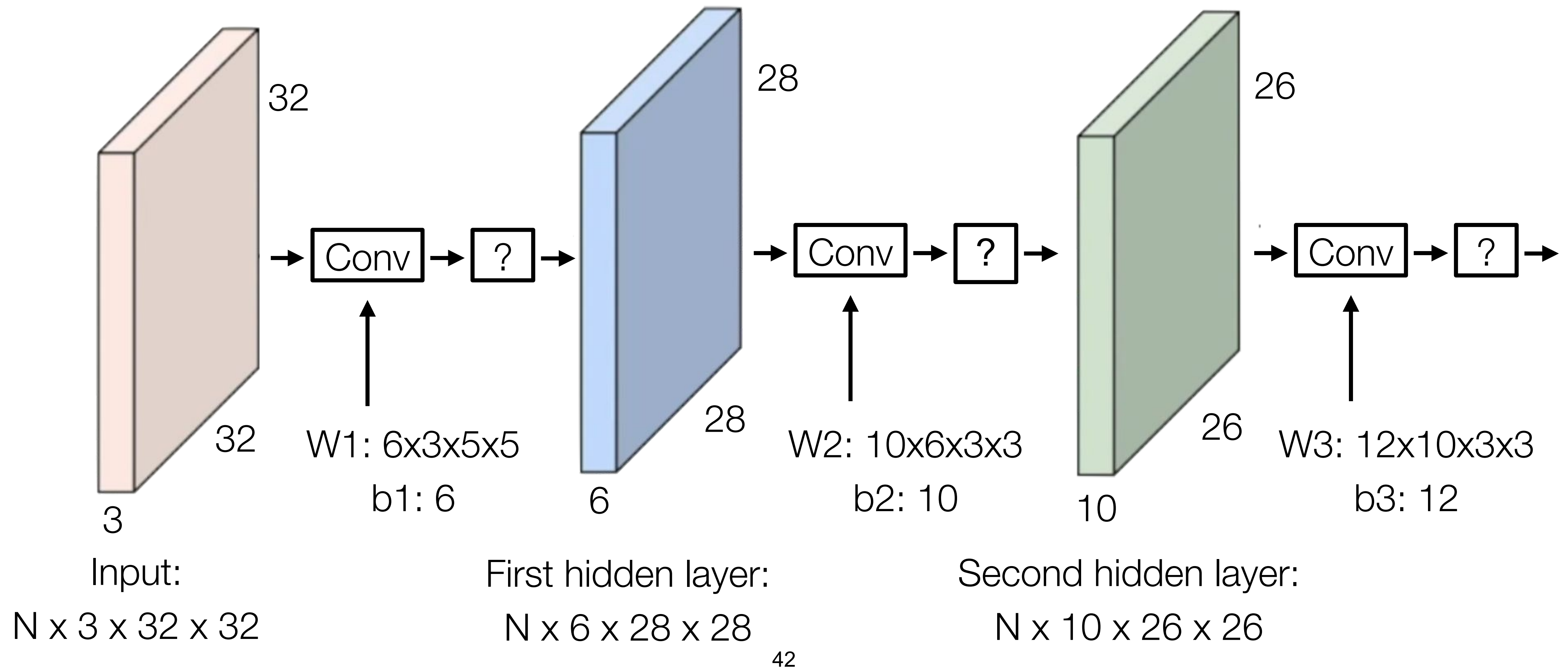
What happens if we stack multiple convolutional layers?

We get another convolution, because the convolution is a linear operation.

The result has the same representational capacity
as another convolutional layer.

To solve this problem: we have to insert a non-linear activation function
layer between the convolutional layers.

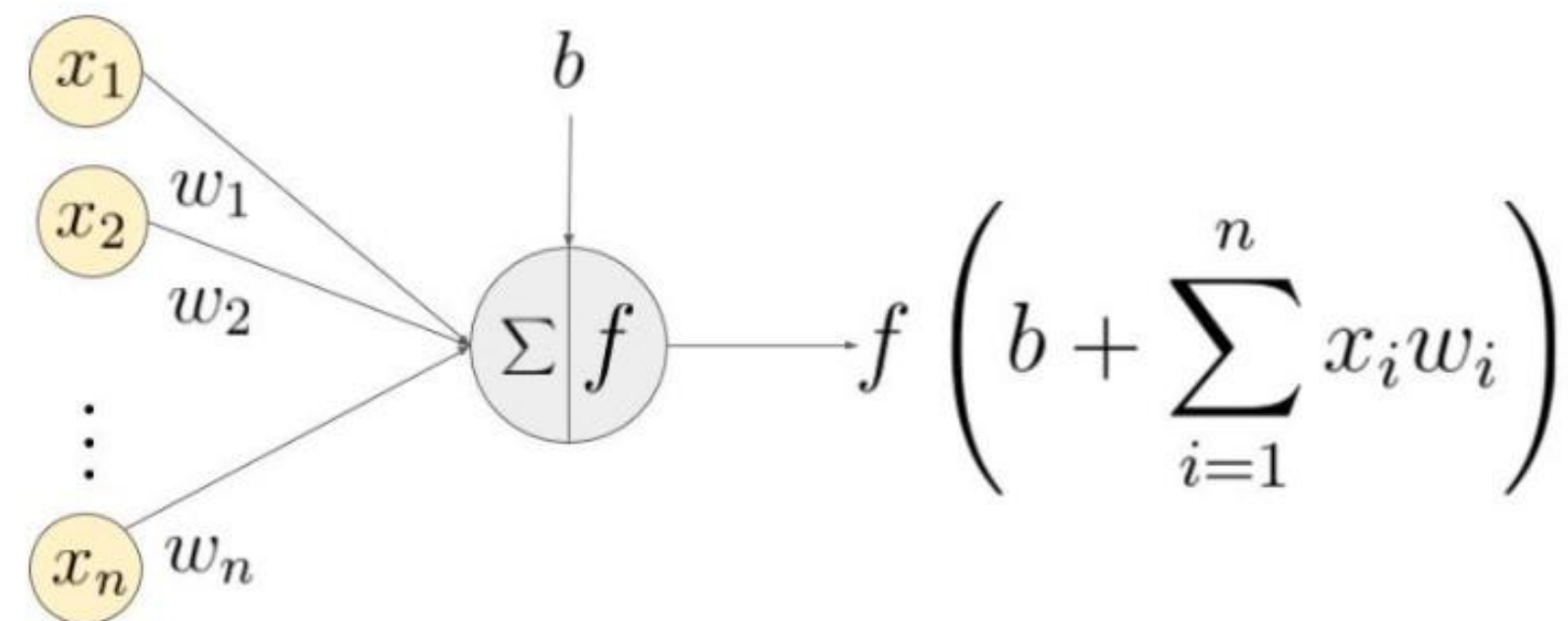
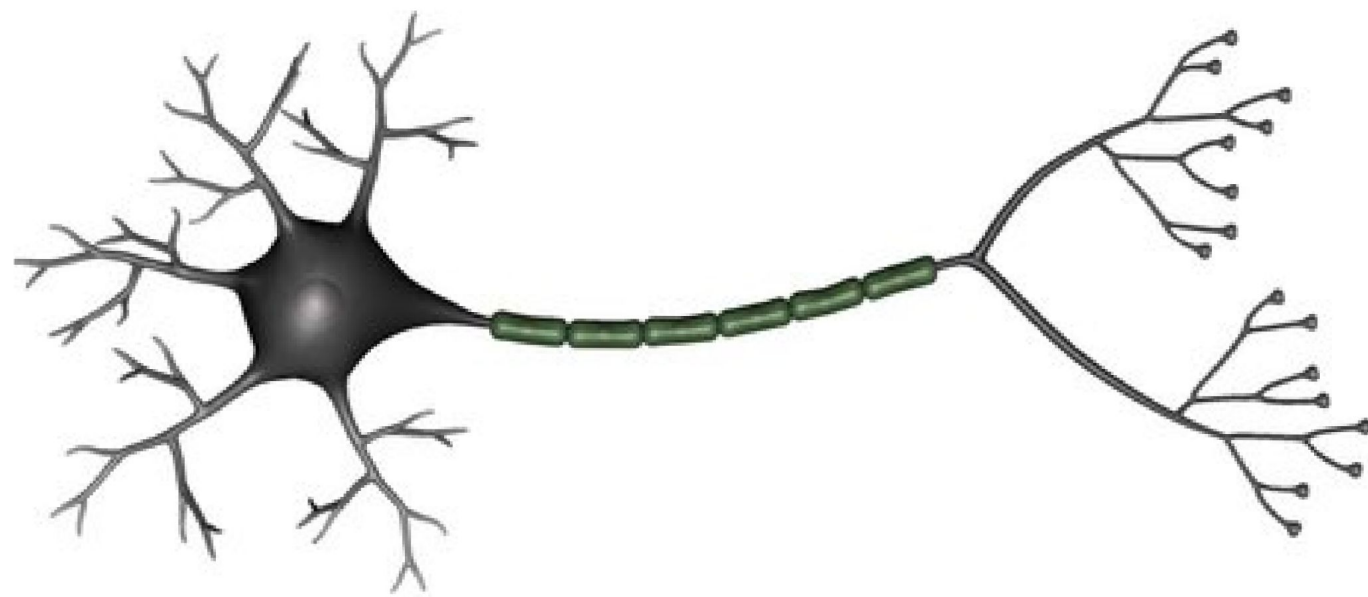
Stacking Convolutions



Activations Functions

Activation functions f are:

- Non-linear functions;
- They are sandwiched between 2 convolutional layers;
- The primary role of an activation function is to enable the neural network to represent and learn complex patterns in the data. Without them, the network would behave like a simple linear model.

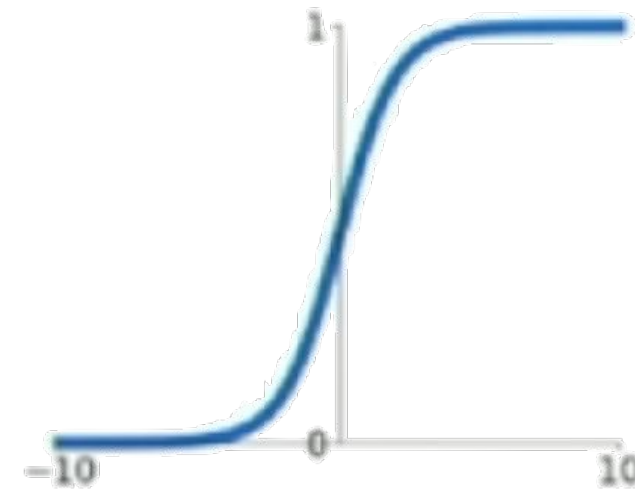


Activations Functions

ReLU is a good default choice for most problems

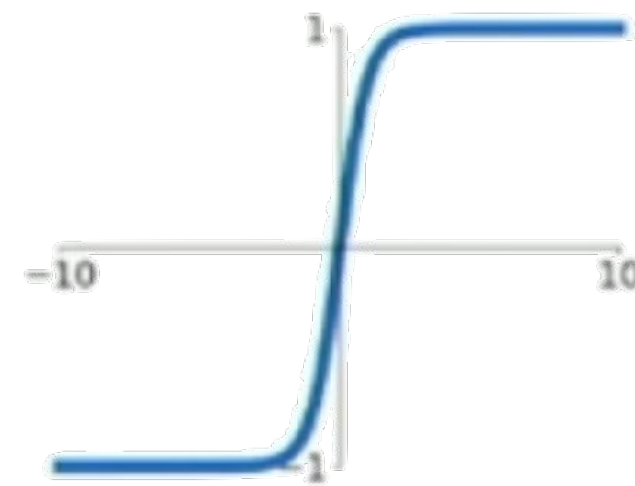
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



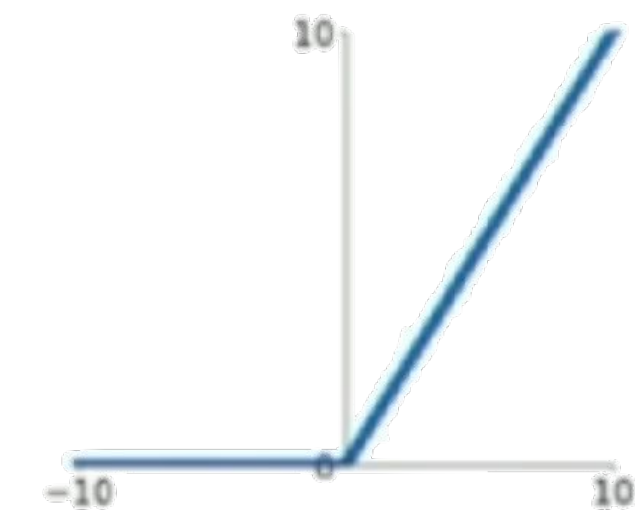
tanh

$$\tanh(x)$$



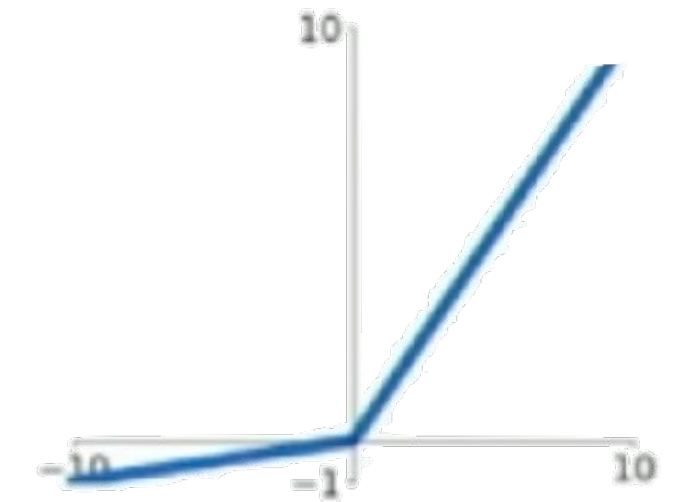
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

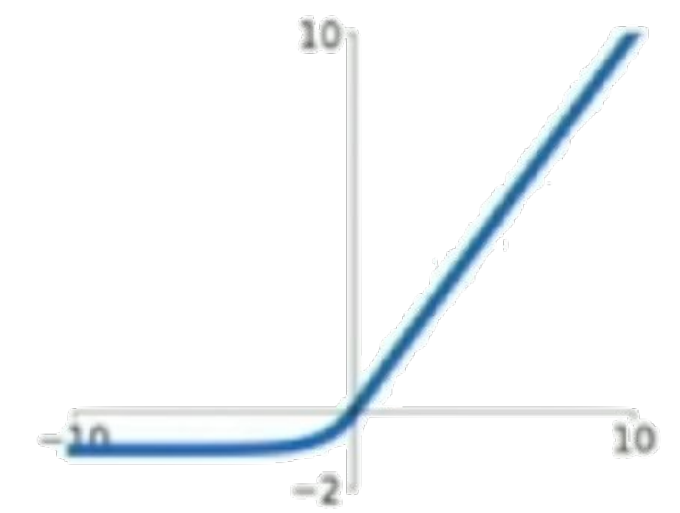


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

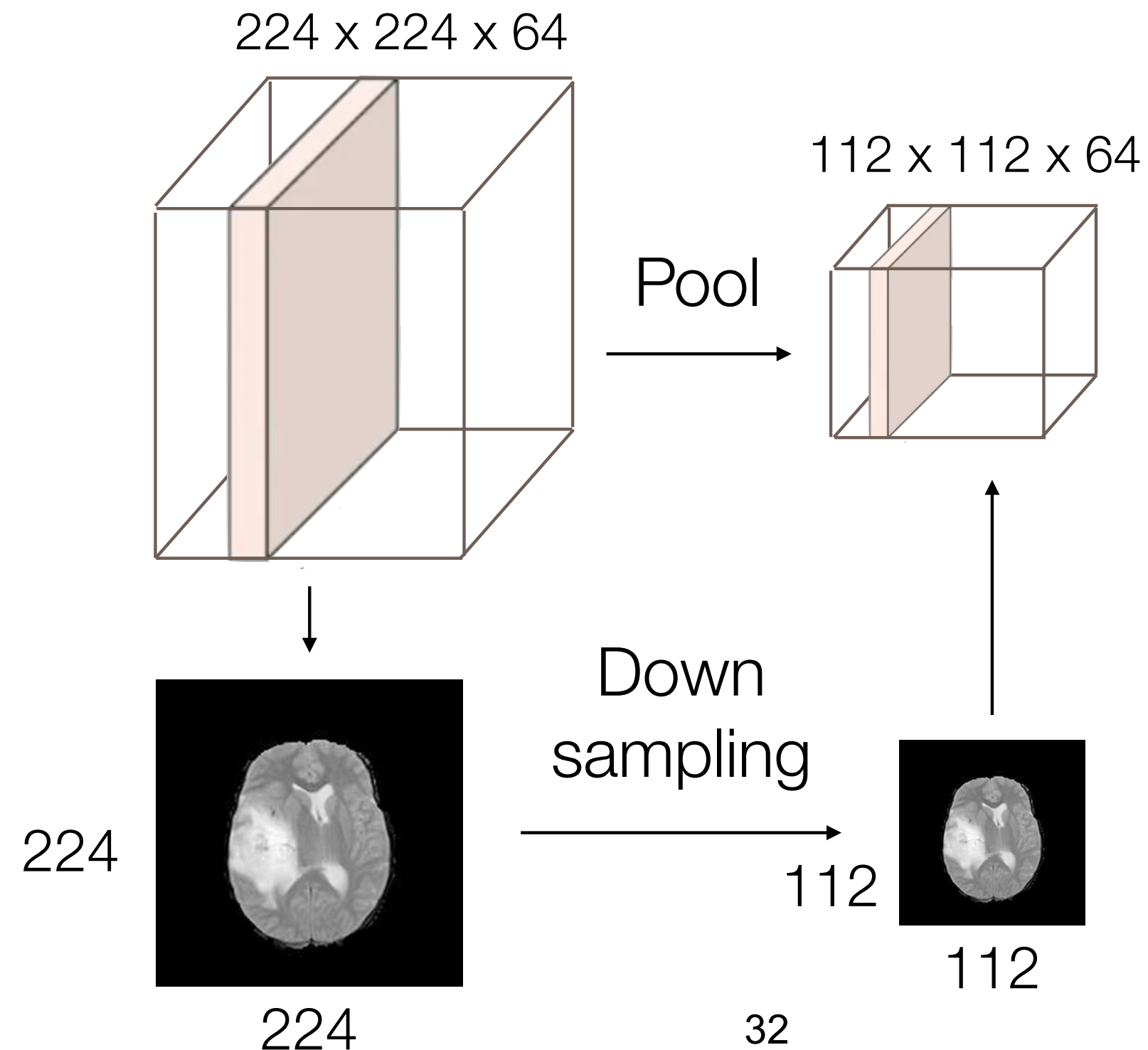
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Pooling Layers

They are a way to downsample the neural network, in a way that does not involve any learnable parameter.



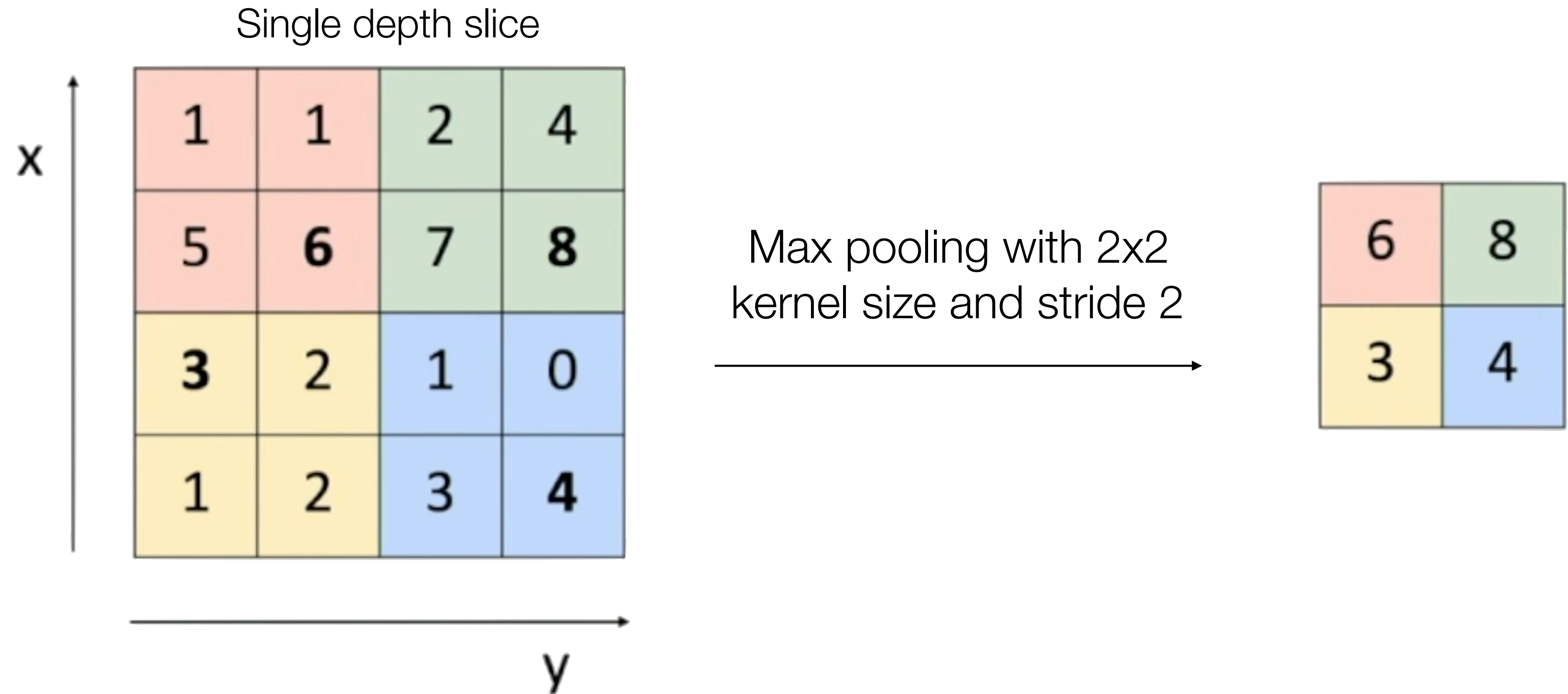
Hyperparameters:

Kernel Size

Stride

Pooling function

Max Pooling



Introduces invariance to small spatial shifts: choosing the maximum, if a thing in the image would have been moved a little, the maximum could still be the same.

Batch normalization layers

Networks converge faster if inputs have been whitened (zero mean, unit variances) and are uncorrelated to account for covariate shift

We can have internal **covariate shift**; normalization could be useful also at the level of hidden layers.

Batch normalization is a technique to cope with this:

- Forces activations to take values on a unit Gaussian at the beginning of the training;
- This layer is added after convolutional layers (or fully connected layers), and before nonlinearities;
- Can be interpreted as doing preprocessing at every layer of the network, but integrated into the network itself in a differentiable way.

Batch normalization layers

Idea: we want to receive some outputs from a previous layer and we want to normalize them such that they have zero-mean and univariate distribution.

Result: layers that see inputs come from a more stationary distribution over the process of training.

Why? Because it reduces the internal covariate shift, which is the variation of the distribution of the outputs from that layers during the training process.

We can normalize a batch of activations like this:

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

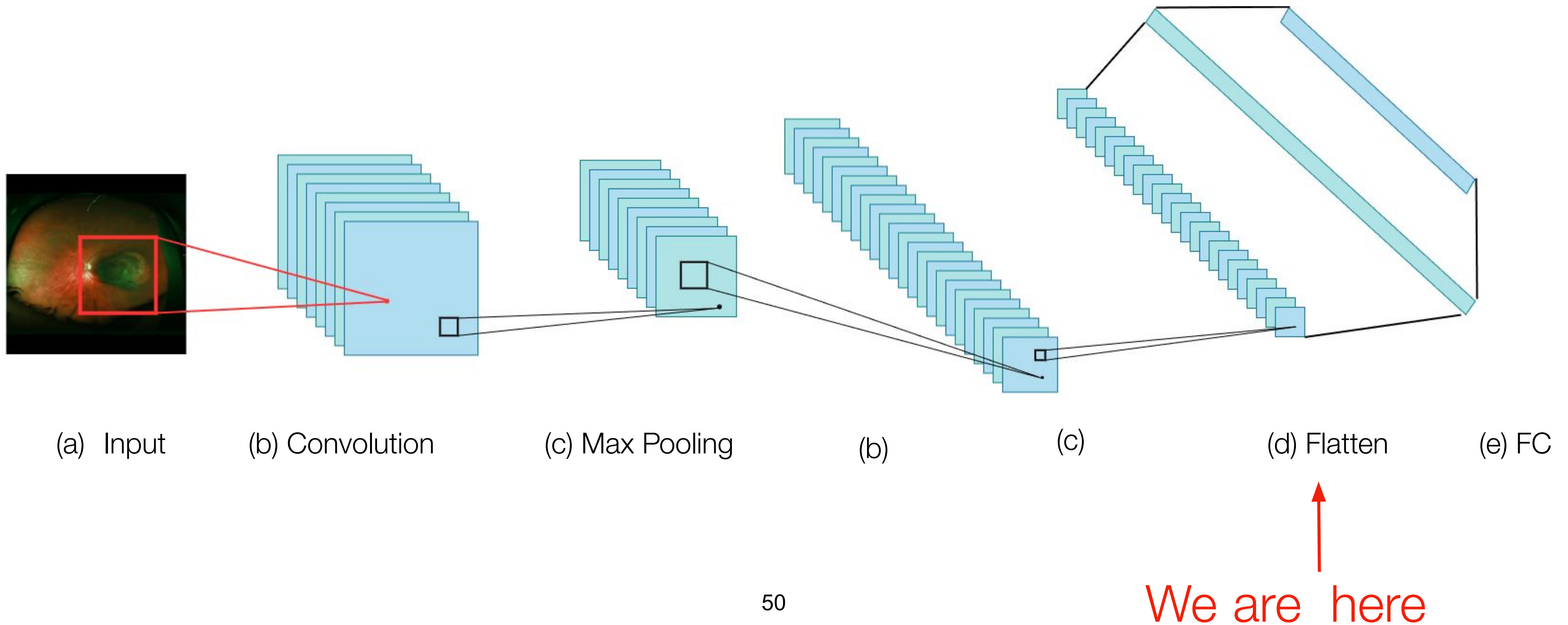
Batch normalization layers

Have shown to

- Improve gradient flow through the network;
- Allow using higher learning rates (therefore, faster training);
- Reduce strong dependence on weights initialization;
- Act as a form of regularization;

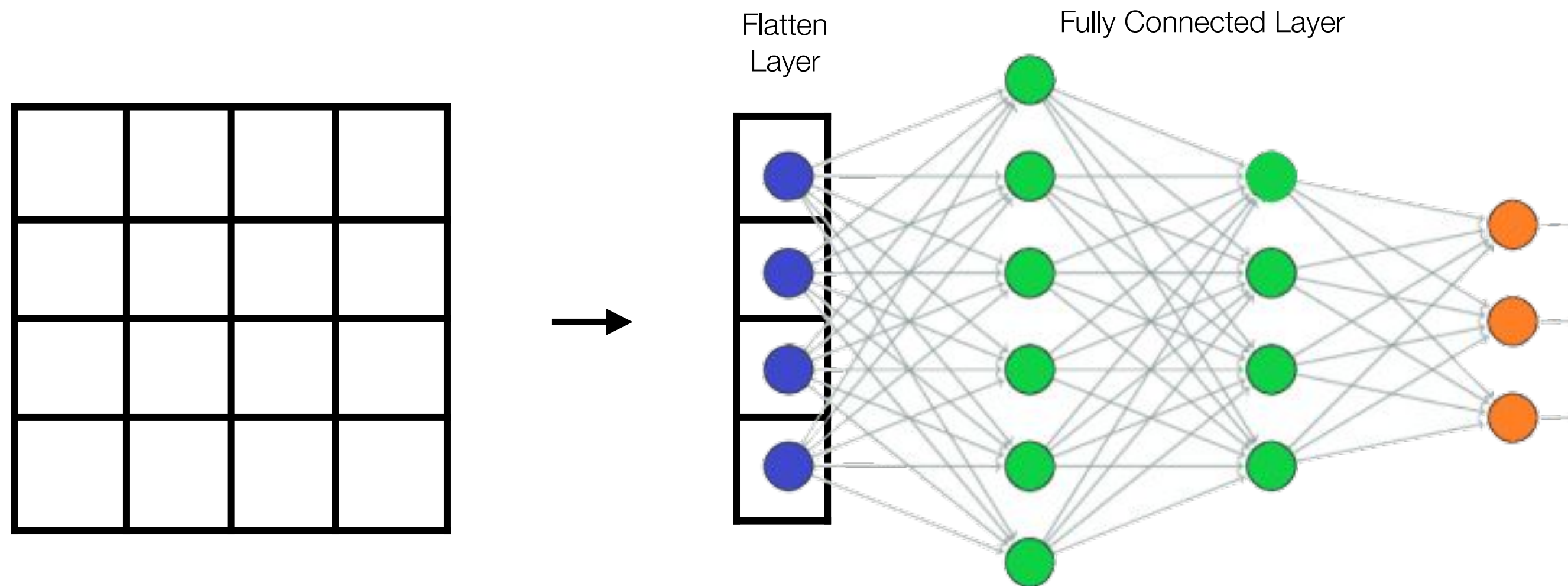
Convolutional Network

Classic architecture: [Conv, ReLU, Pool] x N, **flatten**, [FC, ReLU] x N, **FC**



Flatten Layer

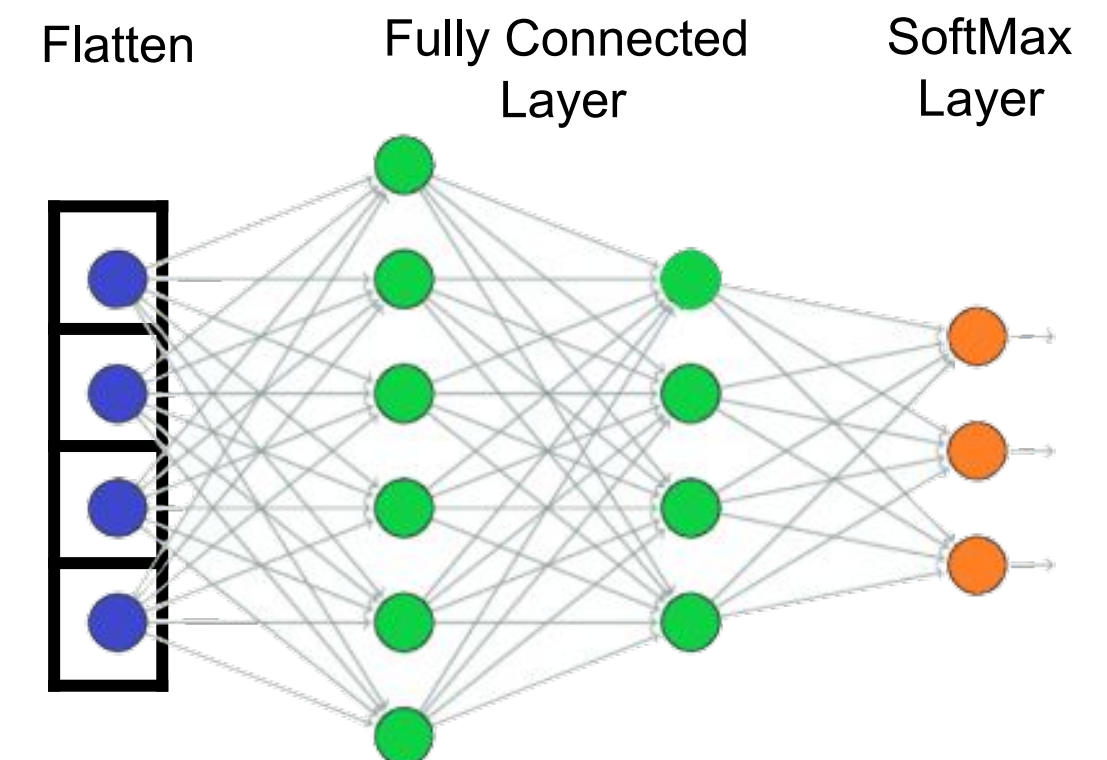
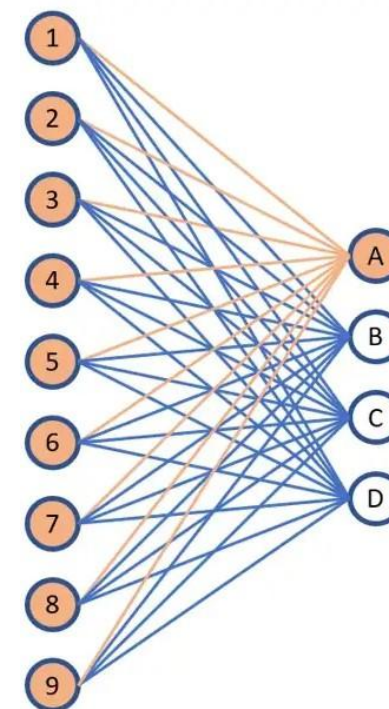
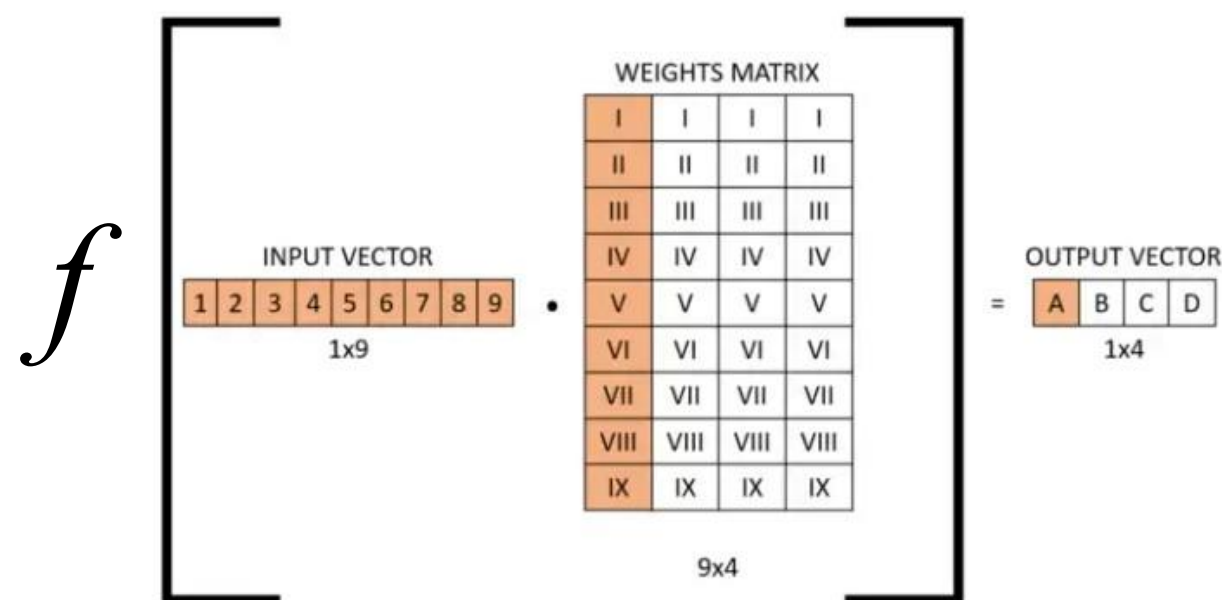
- The flatten layer is used to make the multidimensional input 1D;
- The data coming from flatten layer is passed first to affine function and then to the nonlinear function.



Fully Connected Layers

A fully connected layer is:

- The last layer in the convolutional neural network;
- A combination of affine functions $Wx + b$ (called neuron, or perceptron), and a nonlinear function f (Sigmoid, TanH and ReLu);
- All possible connections layer to layer are present, meaning every input of the input vector influences every output of the output vector.



SoftMax Layers

- The softmax layer transforms the FC layers outputs as that can be interpreted as probabilities;
- The input values can be positive, negative, zero, or greater than one, but the softmax transforms them into values between 0 and 1.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

z: The input vector;

e^{z_i} : The standard exponential function is applied to each element of the input vector. This gives a positive value 0, which will be very small if the input was negative, and very large if the input was large. However, it is still not fixed in the range (0,1) which is what is required of a probability;

$\sum e^{z_j}$: The normalization term. It ensures that all the output values of the function will sum to 1 and each be in the range (0, 1), thus constituting a valid probability distribution;

K: The number of classes in the multi-class classifier

Neural Networks Training

Training Process

$$\hat{y} = f(w^T x + b)$$

Reminder: equation that represents how a neural network processes the input data at each layer and produces a predicted output value. During training the model maps the relationship between the training data and the outputs.

The training process is made of 2 phases:

1. Each training input is loaded into the neural network in a process called **forward propagation**;
2. Once the model has produced an output this predicted output is compared against the given target output in a process called **backpropagation**;

The neural network updates its hyperparameters, the weights, and biases, to satisfy the equation above, and so that it now outputs a result closer to the target output.

Loss Functions

- A loss function is a function that compares the target and predicted output values;
- It measures how well the neural network models the training data;
- When training, we aim to minimize this loss between the predicted and target outputs;
- The hyperparameters such as the weights, the biases are adjusted to minimize the average loss.

$$J(w^T, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

Loss Functions

In supervised learning, there are two main types of loss functions — these correlate to the 2 major types of neural networks: regression and classification loss functions.

1. **Regression Loss Functions:** used in regression neural networks; given an input value, the model predicts a corresponding output value (rather than pre-selected labels);
2. **Classification Loss Functions:** used in classification neural networks; given an input, the neural network produces a vector of probabilities of the input belonging to various pre-set categories, they can then select the category with the highest probability of belonging.

Regression Loss Functions - Mean Square Error

MSE finds the average of the squared differences between the target and the predicted outputs:

- The difference is squared, it does not matter whether the predicted value is above or below the target value; values with a large error are penalized;
- Advantages:
 1. Easy to implement;
 2. Always differentiable because of the square;
 3. Clearly defined global minimum;
- Disadvantage: it is very sensitive to outliers; if a predicted value is significantly greater than or less than its target value, this will significantly increase the loss.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

Classification Loss Functions - Binary Cross Entropy

This is the loss function used in binary classification models, where the model takes in an input and has to classify it into one of two pre-set categories. In these cases, what the network does is to produce a probability.

$P(i)$ = probability of belonging to class 1;

$1 - p(i)$ = probability of belonging to class 0

BCE compares the true label (0 or 1) with the predicted probability and penalizes the model based on how far the prediction is from the actual class.

- Advantage: it is differentiable;
- Disadvantages:
 1. Multiple local minima;
 2. Not very intuitive.

$$CE = \frac{1}{n} \sum_{i=1}^n - (y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i))$$

Classification Loss Functions - Categorical Cross Entropy

In cases where the number of classes is greater than two, we utilize categorical cross-entropy — this follows a very similar process to binary cross-entropy. The model outputs a probability distribution over all M classes using a softmax function, and the loss penalizes the model based on the probability assigned to the correct class.

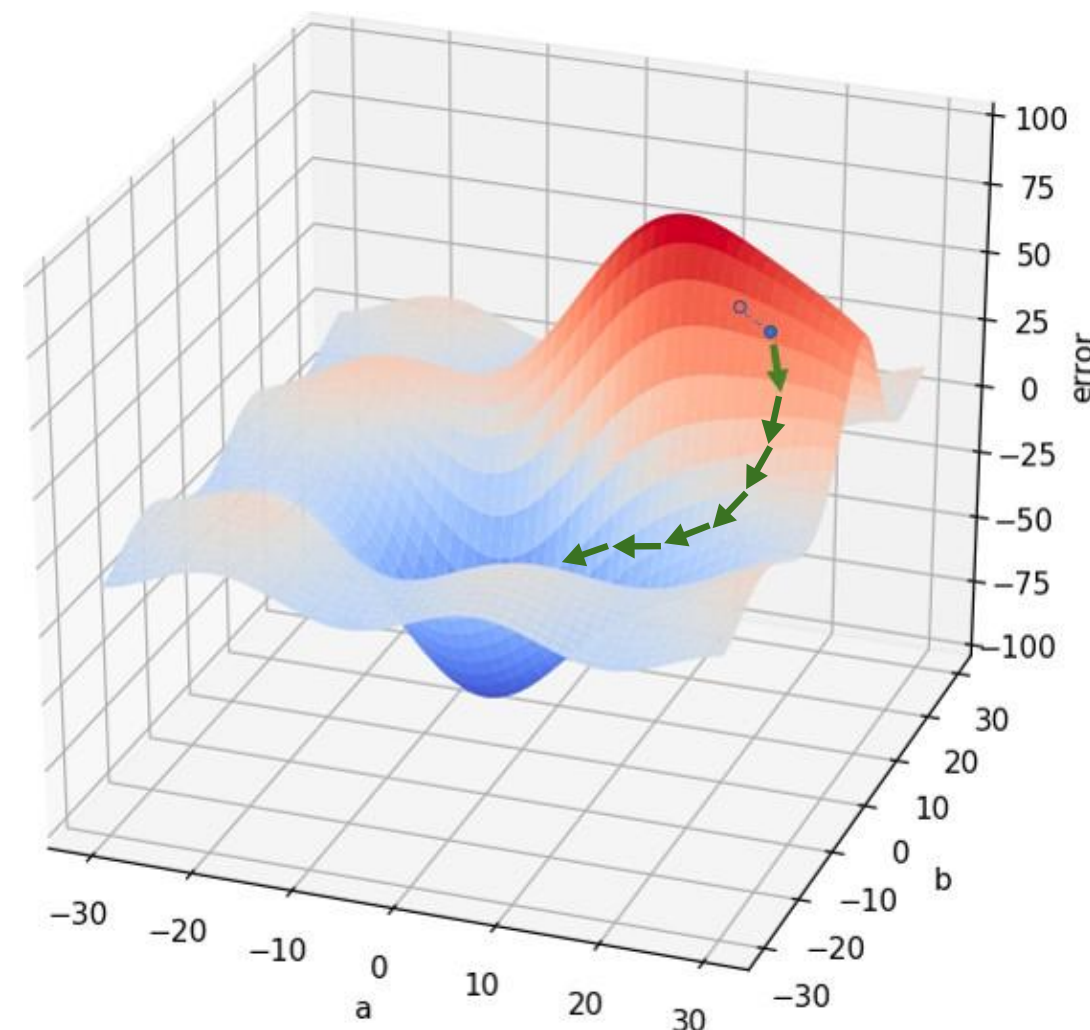
Binary cross-entropy is a special case of categorical cross-entropy, where $M = 2$, which is the number of categories is 2.

$$CE = -\frac{1}{n} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(p_{ij})$$

Loss Functions and NN Optimization

In Deep Learning optimizers are:

- Mathematical functions used to update weights based on the error.
- Functions which are dependent on model's learnable parameters;
- They leverage hyperparameters of a neural networks: e.g., **learning rate**

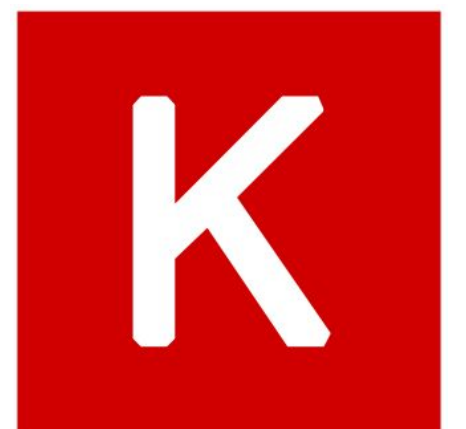


Deep Learning Frameworks

Deep Learning frameworks are high-level software libraries that provide building blocks for designing, training and validating neural networks



They abstract complex mathematical operations, such as gradient updates and backpropagation, and allow easy GPU acceleration.



Project Time

Dataset

FIVES Dataset: <https://www.nature.com/articles/s41597-022-01564-3>

Access dataset here:

[https://figshare.com/articles/figure/FIVES A Fundus Image Dataset for AI-based Vessel Segmentation/19688169](https://figshare.com/articles/figure/FIVES_A_Fundus_Image_Dataset_for_AI-based_Vessel_Segmentation/19688169)

Objective

Preprocess the dataset fixing issues (if different image sizes are present, dataset normalization/standardization) and:

- (a) **classify** each image into one of the four possible classes (diabetic retinopathy, glaucoma, AMD, healthy)
- (b) **Segment** the retinal vessels in each image (we will have a lecture about segmentation)

Motivation

Vessel morphology is linked to multiple clinical conditions, and the retinal vasculature provides an opportunity for its direct observation

Biomedical Computer Vision

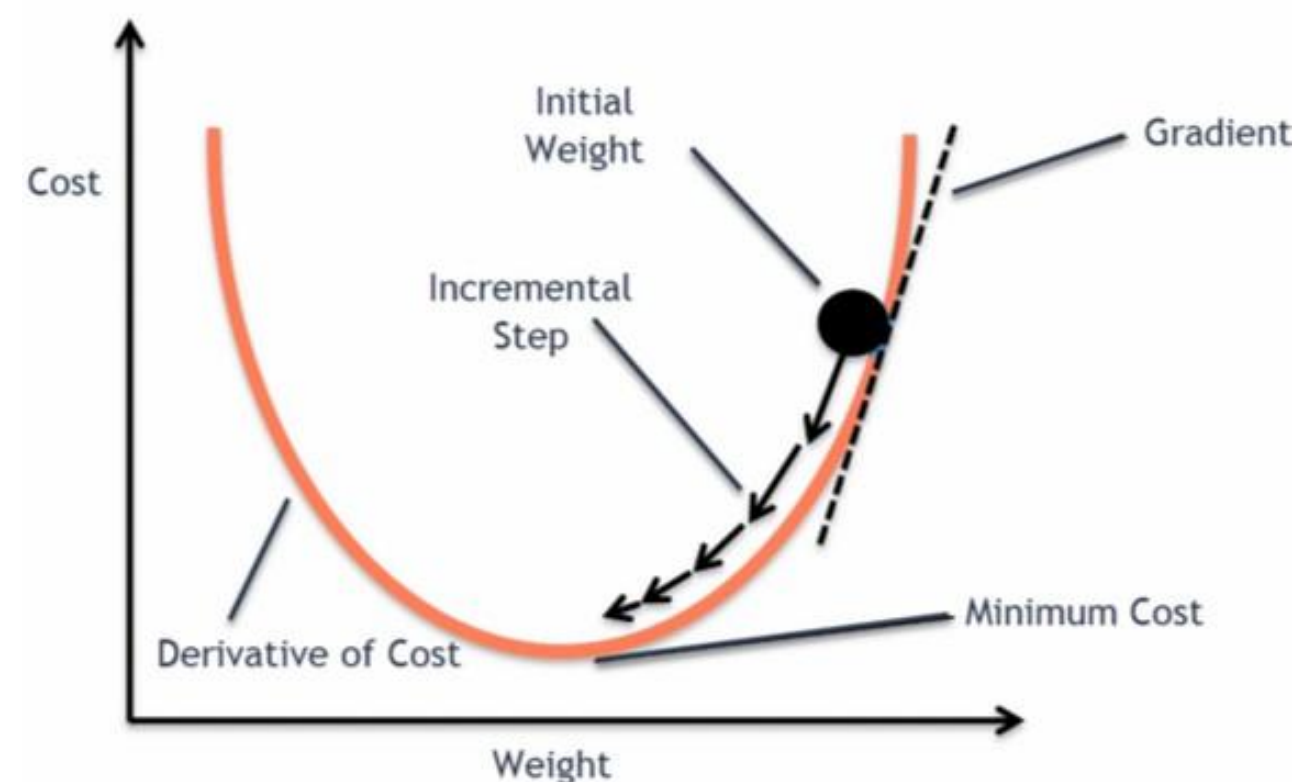
March 31st, 2026

Virginia Tasso
virginia.tasso@polimi.it



Gradient Descent

- Gradient descent is an optimization algorithm based on a convex function and tweaks its parameters iteratively to minimize a given function to its local minimum;
- Gradient descent iteratively reduces a loss function by moving in the direction opposite to that of steepest ascent;
- It is dependent on the derivatives of the loss function for finding minima;
- Advantage: easy to implement and to understand;
- Disadvantage: calculates the gradient for the entire data set in one update, the calculation is very slow and memory expensive.



$$W_{new} = W_{old} - \alpha * \frac{\partial(Loss)}{\partial(W_{old})}$$

↑
Learning rate
(incremental step)

SGD (Stochastic Gradient Descent) with momentum

- Stochastic because it updates the model parameters one by one;
- Momentum η simulates the inertia of an object when it is moving, that is, the direction of the previous update is retained to a certain extent during the update, while the current update gradient is used to fine-tune the final update direction.

$$W_{new} = \eta W_{old} - \alpha * \frac{\partial(Loss)}{\partial(W_{old})}$$

- Advantage: it helps to reduce the noise;
- Disadvantage: extra hyperparameter is added, fixed learning rate.

AdaGrad (Adaptive Gradient Descent)

With AdaGrad we can use different learning rates for each and every neuron for each and every hidden layer based on different iterations.

$$W_{new} = W_{old} - \frac{\alpha}{\sqrt{\epsilon I + \text{diag}(G_t)}} \cdot \frac{\partial(Loss)}{\partial(W_{old})}$$

$$G_t = \sum_{\tau=1}^t g_{\tau} g_{\tau}^T$$

where w is the parameter to be updated, α is the initial learning rate, ϵ is some small quantity that used to avoid the division of zero, I is the identity matrix, and G_t is the sum of the outer product of the gradients until time-step t .

- Advantage: learning rate changes adaptively with iterations;
- Disadvantage:
 1. If the neural network is deep the learning rate becomes very small number which will cause dead neuron problem;
 2. The sum of the gradient squared only grows and never shrinks.

RMS-Prop (Root Mean Square Propagation)

- RMS-Prop is a special version of Adagrad, and it solves its disadvantages;
- It uses a decaying average of partial gradients in the adaptation of the step size for each parameter;

$$W_{new} = W_{old} - \frac{\alpha}{\sqrt{\epsilon I + \text{diag}(G_{t_{new}})}} \cdot \frac{\partial(Loss)}{\partial(W_{old})} \quad G_{t_{new}} = G_{t_{old}} \cdot \gamma + \left(\frac{\partial(Loss)}{\partial(W_{old})}\right)^2 \cdot (1 - \gamma)$$

where w is the parameter to be updated, α is the initial learning rate, ϵ is some small quantity that used to avoid the division of zero, I is the identity matrix, and $G_{t_{new}}$ is the sum of the outer product of the gradients until time-step t .

- Advantage: it chooses a different learning rate for each parameter.
- Disadvantage: slow learning.

Adam

- Adam can be looked at as a combination of RMS-Prop and SGD with momentum;
- It uses the squared gradients to scale the learning rate like RMS-Prop;
- It takes advantage of momentum by using moving average of the gradient instead of gradient itself like SGD with momentum;
- Adam uses estimations of first (mean m_t) and second moments (uncentered variance v_t) of gradient to adapt the learning rate for each weight of the neural network;
- Advantage: computationally and memory efficient.

$$w_t = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad m_t = (1 - \beta_1) \sum_{i=0}^t \beta_1^{t-i} g_i \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad v_t = (1 - \beta_2) \sum_{i=0}^t \beta_2^{t-i} g_i^2$$